



Servicios RESTful

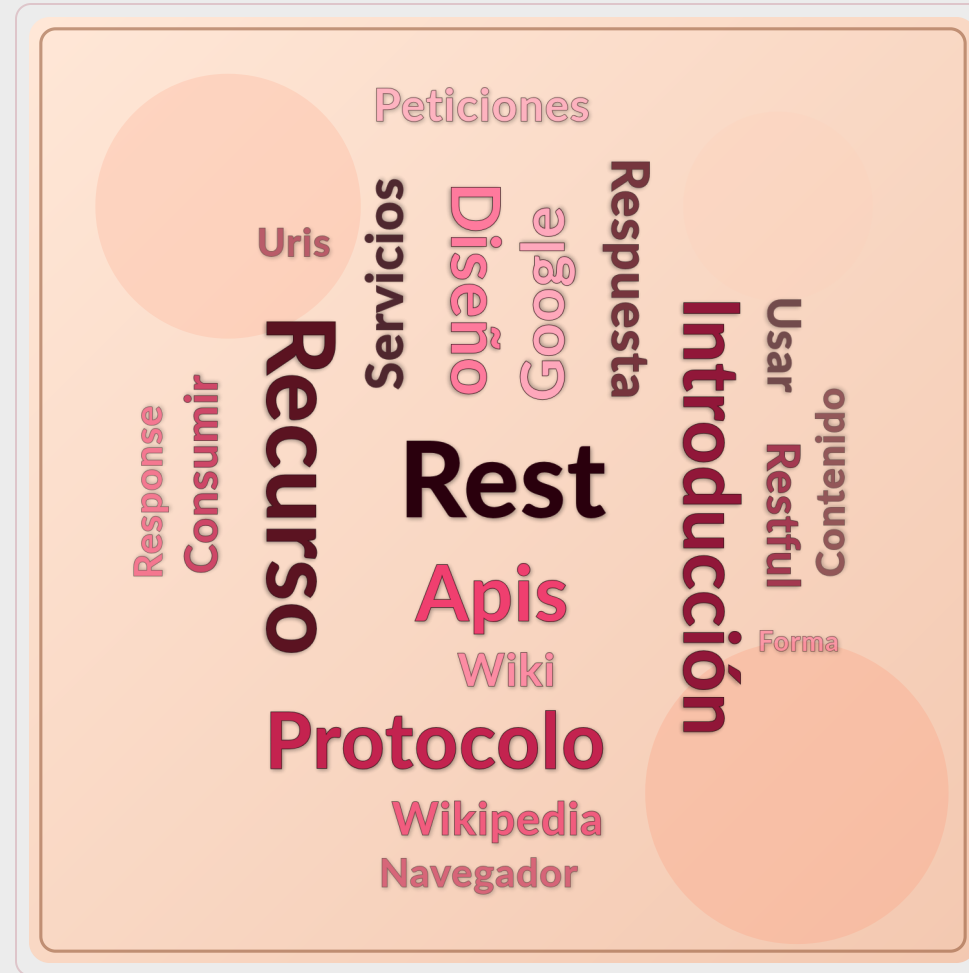
Universidad de Sevilla

Departamento de Lenguajes y Sistemas
Informáticos

Carlos Arévalo, Daniel Ayala, José Calderón,
Margarita Cruz, Inma Hernández, David Ruiz

UNIVERSIDAD DE SEVILLA

Objetivos



Contenido

Introducción

URIs

Protocolo HTTP

Diseño de APIs REST

Consumir APIs REST



Contenido

Introducción

URIs

Protocolo HTTP

Diseño de APIs REST

Consumir APIs REST



Introducción: API

Application Programming Interface

Forma en la que los programas y/o sitios Web intercambian datos (formato JSON o XML)

Sirven para:

- Ofrecer datos y servicios a aplicaciones externas (móvil, Web, consola, ...)
- Ofrecer datos y servicios a otros desarrolladores
- Consumir datos de otros servicios (Twitter, Google, Facebook, YouTube, Amazon, Discord, etc.)

<https://rapidapi.com/collection/list-of-free-apis>



Introducción: API

JSON vs XML

```
http://localhost:8080/Json/SyncReply/Contacts
{
  - Contacts: [
    - {
      FirstName: "Demis",
      LastName: "Bellot",
      Email: "demis.bellot@gmail.com"
    },
    - {
      FirstName: "Steve",
      LastName: "Jobs",
      Email: "steve@apple.com"
    },
    - {
      FirstName: "Steve",
      LastName: "Ballmer",
      Email: "steve@microsoft.com"
    },
    - {
      FirstName: "Eric",
      LastName: "Schmidt",
      Email: "eric@google.com"
    },
    - {
      FirstName: "Larry",
      LastName: "Ellison",
      Email: "larry@oracle.com"
    }
  ]
}

http://localhost:8080/Xml/SyncReply/Contacts
<ContactsResponse xmlns:i="http://www.w3.org/20
  <Contacts>
    <Contact>
      <Email>demis.bellot@gmail.com</Email>
      <FirstName>Demis</FirstName>
      <LastName>Bellot</LastName>
    </Contact>
    <Contact>
      <Email>steve@apple.com</Email>
      <FirstName>Steve</FirstName>
      <LastName>Jobs</LastName>
    </Contact>
    <Contact>
      <Email>steve@microsoft.com</Email>
      <FirstName>Steve</FirstName>
      <LastName>Ballmer</LastName>
    </Contact>
    <Contact>
      <Email>eric@google.com</Email>
      <FirstName>Eric</FirstName>
      <LastName>Schmidt</LastName>
    </Contact>
    <Contact>
      <Email>larry@oracle.com</Email>
      <FirstName>Larry</FirstName>
      <LastName>Ellison</LastName>
    </Contact>
  </Contacts>
</ContactsResponse>
```

Introducción: REST

La World Wide Web (WWW, W³, la Web) es un sistema de documentos de hipertexto interconectados accesibles a través de Internet.

En el año 2000, **Roy T. Fielding** estudió en su tesis por qué la Web había sido tan exitosa y enunció una serie de principios generales que generalizaban su arquitectura: **RE**presentational **ST**ate **T**ransfer (REST).

REST es un estilo arquitectónico para sistemas distribuidos basados en contenidos hipermedia como la Web y operaciones sin estado.

El principal beneficio de REST es la interoperabilidad entre sistemas en la Web

<http://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>

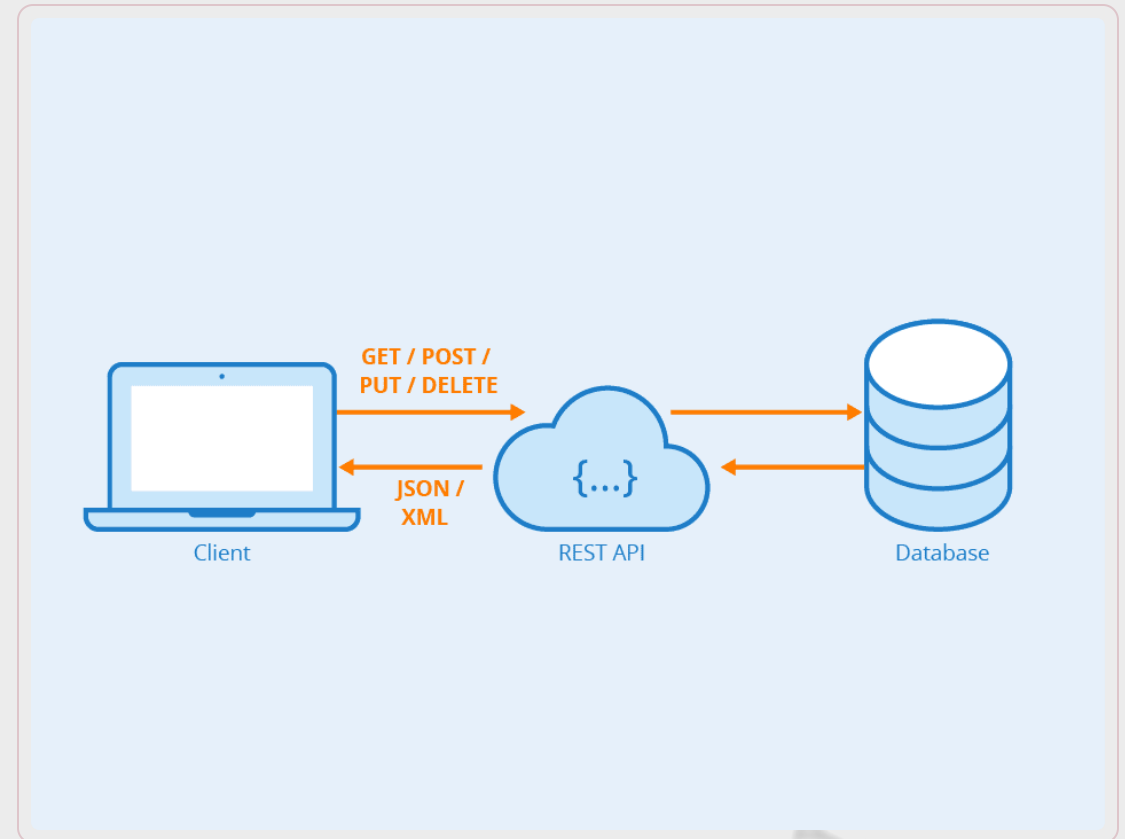
https://en.wikipedia.org/wiki/Representational_state_transfer



Introducción: API RESTful

API que sigue los principios REST

- Llamadas a través de peticiones HTTP
- Métodos HTTP según la operación
- Resultado como código de estado HTTP
- Interfaz uniforme
- Peticiones sin estado
- Cacheable
- Separación cliente y servidor



Introducción: Principios REST

Recurso: cualquier cosa que expongamos en la Web, una imagen, un vídeo, un libro, un documento HTML, etc.

URI: (Uniform Resource Identifier) identifica de manera única a un recurso web y, al mismo tiempo, lo hace direccionable y manipulable utilizando un protocolo de aplicación como HTTP



Contenido

Introducción

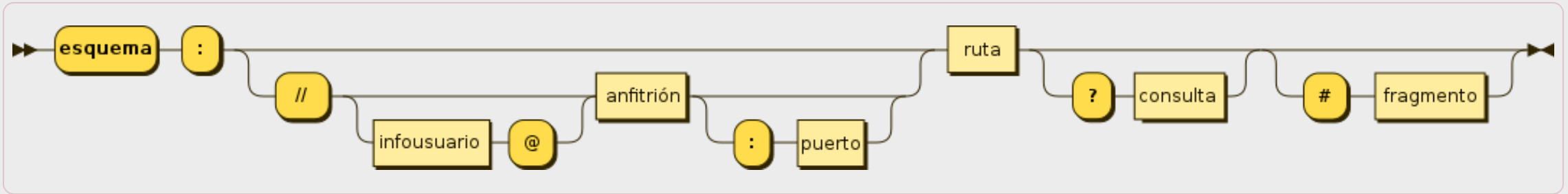
URIs

Protocolo HTTP

Diseño de APIs REST

Consumir APIs REST

Introducción: URI



https://mariadb.com:443/kb/en/library/mysql-command-line-client/#the-mysql_history-file

Esquema: protocolo de acceso (http, https, ftp, ftps, smtp, ...)

Host: identificación jerárquica que asigna la autoridad de nombres.

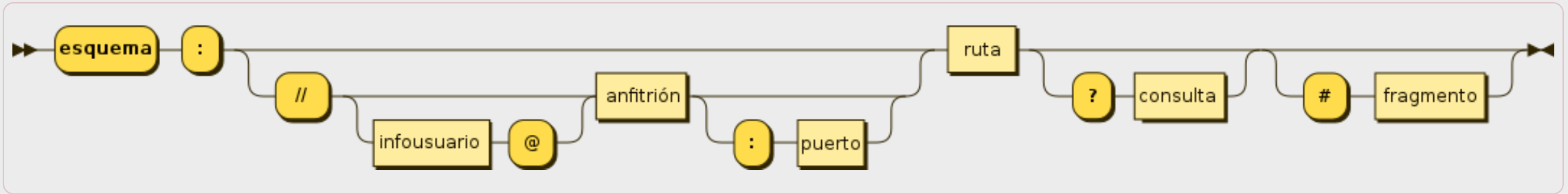
Puerto: número entre 0 y 2¹⁶-1 (65 535) (puertos conocidos)

Ruta: identifica al recurso en el ámbito del anfitrión.

Consulta: Pares ?Clave1=Valor1&Clave2=Valor2... que identifica al recurso de forma paramétrica

Fragmento: Parte de un recurso

Introducción: URI



<https://calendar.google.com/calendar/r/week?tab=mc>

Esquema: protocolo de acceso (http, https, ftp, ftps, smtp, ...)

Host: identificación jerárquica que asigna la autoridad de nombres.

Puerto: número entre 0 y 2¹⁶-1 (65 535) (puertos conocidos)

Ruta: identifica al recurso en el ámbito del anfitrión.

Consulta: Pares ?Clave1=Valor1&Clave2=Valor2... que identifica al recurso de forma paramétrica

Fragmento: Parte de un recurso

Introducción: Principios REST

Un recurso puede tener varias URI

- <http://www.google.com>
- <https://www.google.com>
- <http://google.com>

Un recurso puede tener varias representaciones

- XHTML , HTML , TXT , XML , JSON , JPEG , ...

Un recurso puede tener enlaces a otros recursos (HATEOAS)

- <https://en.wikipedia.org/wiki/HATEOAS>

Contenido

Introducción

URIs

Protocolo HTTP

Diseño de APIs REST

Consumir APIs REST

HTTP: Conceptos básicos

Internet: conjunto de redes de comunicación que permite conectar dispositivos usando el protocolo TCP/IP

WWW: servicio dentro de Internet para poder visualizar páginas Web.

HTML: lenguaje utilizado para describir páginas Web.

HTTP: protocolo de nivel (OSI) de aplicación sobre TCP/IP que permite transferencias de información en la WWW

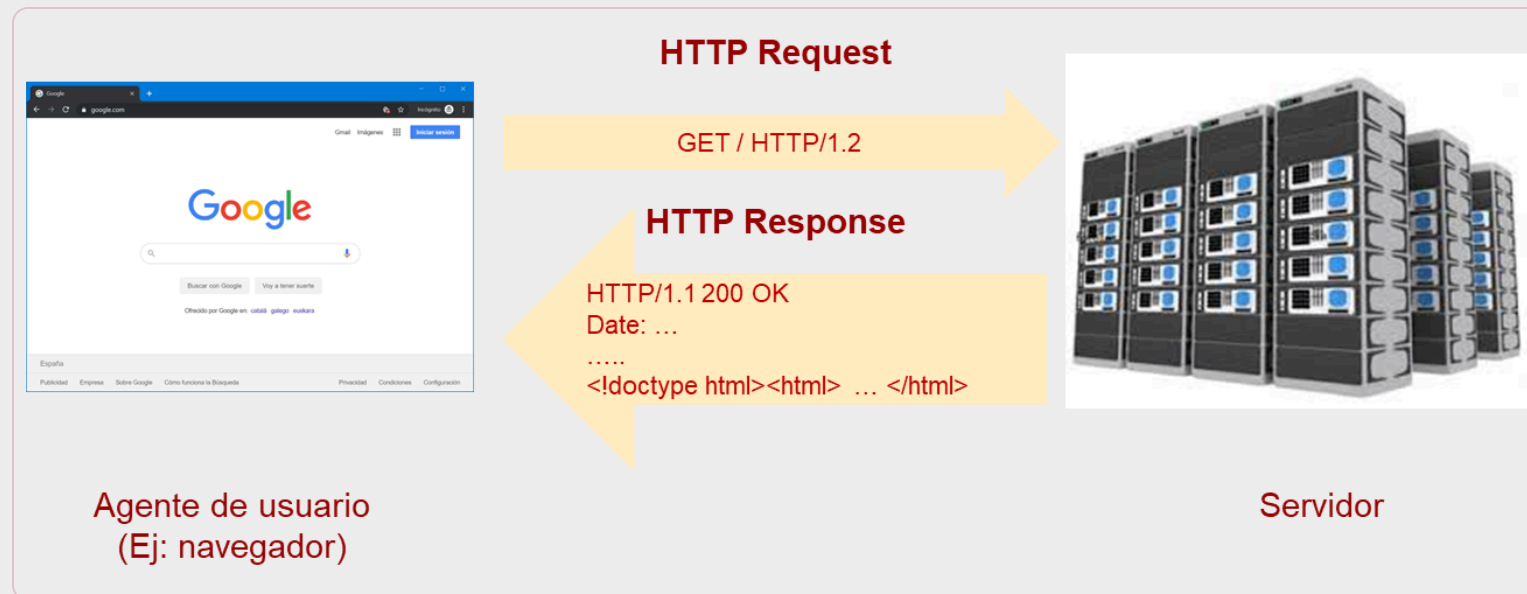


HTTP: Conceptos básicos

Hypertext Transfer Protocol o HTTP es el protocolo de aplicación estándar usado para la comunicación en la Web.

Sigue el esquema petición-respuesta entre un cliente y un servidor.

HTTPS funciona bajo una capa de cifrado pero los mensajes son idénticos



HTTP: El navegador

¿Qué ocurre en el navegador cuando solicitamos una página web (F12)?
www.google.com -> 57 peticiones HTTP

The image shows the Google homepage in a dark theme. The Google logo is centered, with a search bar below it. The search bar contains a plus icon, a microphone icon, and a camera icon. Below the search bar are two buttons: "Buscar con Google" and "Voy a tener suerte". Below these buttons is a link: "Ofrecido por Google en: català galego euskara". At the bottom of the page, there is a footer with the text "España" and a green leaf icon followed by the text "Tres décadas de acción climática: descubre lo que estamos haciendo".

On the right side of the image, the Chrome DevTools network tab is open. It shows a list of 57 HTTP requests. The table has columns for "Nombre", "Estado", "Tipo", "Iniciador", "Tamaño", and "Tiem...". The requests are filtered by "Fetch/XHR". The first request is "m=3yrg4r100w13y10qg3...", which is a script. The second request is "GetAsyncData", which is an xhr. The third request is "m=aLufP?xjs=s4", which is a script. The fourth request is "m=1000Vd,sy91,P6sQOc?xj...", which is a script. The fifth request is "log?format=json&hasfast=...", which is a preflight. The sixth request is "log?format=json&hasfast=...", which is an xhr. The seventh request is "gen_204?ei=1kl3abFYDZL9p...", which is a ping. The eighth request is "save?continue=https://ww...", which is an xhr. The ninth request is "gen_204?ei=1kl3abFYDZL9p...", which is a ping. The tenth request is "gen_204?atyp=i&ct=bxjs&...", which is a text/html. The eleventh request is "gen_204?atyp=csi&ei=1kl3...", which is a ping. The twelfth request is "gen_204?atyp=i&ei=1kl3ab...", which is a ping. The thirteenth request is "gen_204?atyp=i&ei=1kl3ab...", which is a ping. The table continues with more requests, all of which are pings or scripts. The bottom of the table shows the summary: "Solicitudes: 57", "Se ha transferido 924 kB", "Recursos: 2,7 MB", "Finalizar: 24,92 s", and "DOMCon".

HTTP: El navegador

¿Qué ocurre en nuestro navegador cuando solicitamos una página web?

www.us.es -> 104 peticiones HTTP (requests)

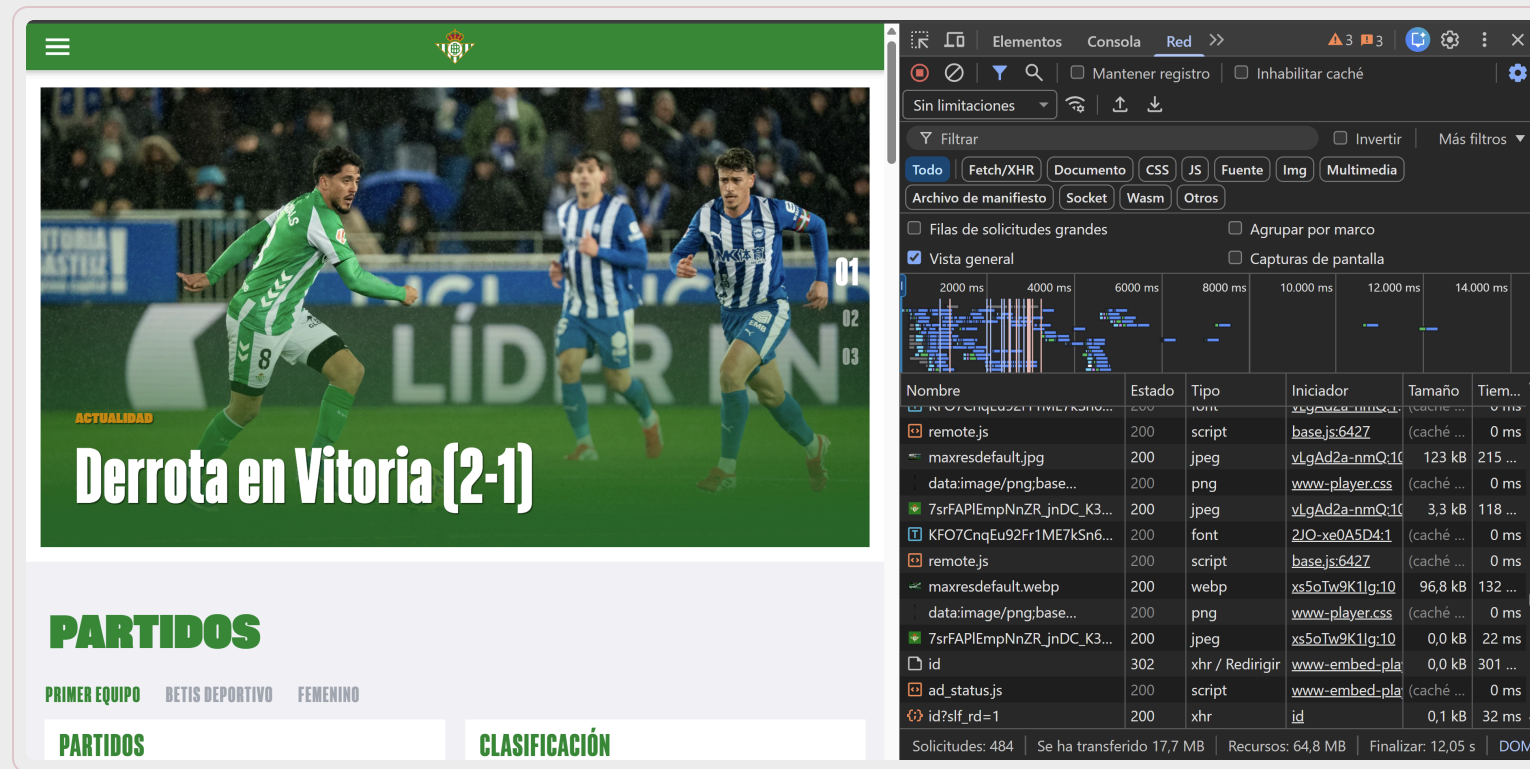
The image shows a screenshot of the University of Seville (US) website and its network traffic in a browser's developer tools. The website features a large banner for 'TV Hall US Enero 2026' and a prominent text overlay: 'Disponible para descarga el TV Hall de enero'. The developer tools panel on the right displays the 'Red' (Network) tab, showing a list of 104 requests. The table below provides a detailed view of these requests.

Nombre	Estado	Tipo	Iniciador	Tamaño	Tiem...
core.usgorg.ee.js	200	script	page.js	27,0 KB	26 ms
b8dbc7c64047561cb95a8d...	200	document		(caché ...)	8 ms
sm.25.html	200	document		1,0 kB	35 ms
flexslider-icon.woff	200	font	css_Rh46ZsQKVf	1,4 kB	1,56 s
data:image/svg+xml;...	200	svg+xml	Otros	(caché ...)	0 ms
data:image/svg+xml;...	200	svg+xml	Otros	(caché ...)	0 ms
data:image/svg+xml;...	200	svg+xml	Otros	(caché ...)	0 ms
data:image/svg+xml;...	200	svg+xml	Otros	(caché ...)	0 ms
data:image/svg+xml;...	200	svg+xml	Otros	(caché ...)	0 ms
data:image/svg+xml;...	200	svg+xml	Otros	(caché ...)	0 ms
widgets.js?_=1769425257054	200	script	js_c8LbWbHV1tx	28,0 kB	86 ms
US-favicon-64x64.png	200	png	Otros	3,2 kB	24 ms
widget_iframe.2f70fb173b9...	200	document		106 kB	114 ...
settings?session_id=f39b32...	200	fetch	widget_iframe.2f	1,0 kB	229 ...

Solicitudes: 104 | Se ha transferido 6,5 MB | Recursos: 8,6 MB | Finalizar: 7,62 s | DOMC...

HTTP: El navegador

¿Qué ocurre en nuestro navegador cuando solicitamos una página web?
www.realbetisbalompie.es -> 484 peticiones HTTP (requests)



The image shows a screenshot of the Real Betis website and the Chrome DevTools Network tab. The website displays a match result: "Derrota en Vitoria (2-1)". The DevTools Network tab shows a list of 484 requests, including various scripts, images, and fonts.

Nombre	Estado	Tipo	Iniciador	Tamaño	Tiem...
remote.js	200	script	base.js:6427	(caché ...)	0 ms
maxresdefault.jpg	200	jpeg	VLgAd2a-nmQ:10	123 kB	215 ...
data:image/png;base...	200	png	www-player.css	(caché ...)	0 ms
7srFAPLEmpNnZR_jnDC_K3...	200	jpeg	VLgAd2a-nmQ:10	3,3 kB	118 ...
KFO7CnqEu92Fr1ME7kSn6...	200	font	2JO-xe0A5D4:1	(caché ...)	0 ms
remote.js	200	script	base.js:6427	(caché ...)	0 ms
maxresdefault.webp	200	webp	xs5oTw9K1lg:10	96,8 kB	132 ...
data:image/png;base...	200	png	www-player.css	(caché ...)	0 ms
7srFAPLEmpNnZR_jnDC_K3...	200	jpeg	xs5oTw9K1lg:10	0,0 kB	22 ms
id	302	xhr / Redirigir	www-embed-pla	0,0 kB	301 ...
ad_status.js	200	script	www-embed-pla	(caché ...)	0 ms
id?slf_rd=1	200	xhr	id	0,1 kB	32 ms

Solicitudes: 484 | Se ha transferido 17,7 MB | Recursos: 64,8 MB | Finalizar: 12,05 s | DOM...

Ayuda con IA (Google Chrome)

The image shows a screenshot of the University of Seville website (www.us.es) with the Chrome DevTools interface open. The website features a header with the university logo and navigation links, a main image of two divers underwater, and a footer with a cookie consent banner. The DevTools interface is in dark mode, with the 'Network' tab selected. A yellow box highlights the 'Debug with AI' button in the top right of the network panel. Another yellow box highlights the AI assistance panel at the bottom, which displays the message: 'Analyzing network data'. The AI panel also shows a detailed explanation of a network request: 'This request is for fetching the Google Tag Manager (GTM) JavaScript library, which is essential for implementing Google Analytics and other tracking functionalities on the web. The id=UA-165949399-1 parameter in the URL specifies the Google Analytics tracking ID.'

UNIVERSIDAD DIGITAL

UNIVERSIDAD DE SEVILLA 1905

LA US ESTUDIAR INVESTIGAR VIVIR LA US EMPRESAS INTERNACIONAL TRABAJA EN LA US

INFORMACIÓN PARA MI

ipulsan el primer corredor de coral de Europa

vilidad internacional. 26-27 Ulyseus, garantía de futuro Uso de la marca visual

Utilizamos cookies propias para el correcto funcionamiento de la página web y de todos sus servicios, y de terceros para analizar el tráfico en nuestra página web. Si continúas navegando, consideramos que aceptas su uso. [Más info](#)

Aceptar

Elements

Clear browser cache

Clear browser cookies

Copy

Filter

All Fetch/XHR Do

Other

Big request rows

Overview

200 ms 400 ms

Override headers

Override content

Show all overrides

Name

Save as...

rs Payload Preview Response

www.us.es

js?id=UA-165949399

css_Rh46ZsQKVfIOHdvamnV...

css_ldp2qOsHlyDSmEqLoKE...

bootstrap.min.css

css_z7cQCB-htM7WX7anLo

100 requests 637 kB transferred

Console What's new AI assistance X Issues

Send feedback

Debug with AI

Start a chat

Request URL

Request Method

Status Code

Referrer Policy

Explain purpose

Explain failures

Assess security headers

7 Internal Redir

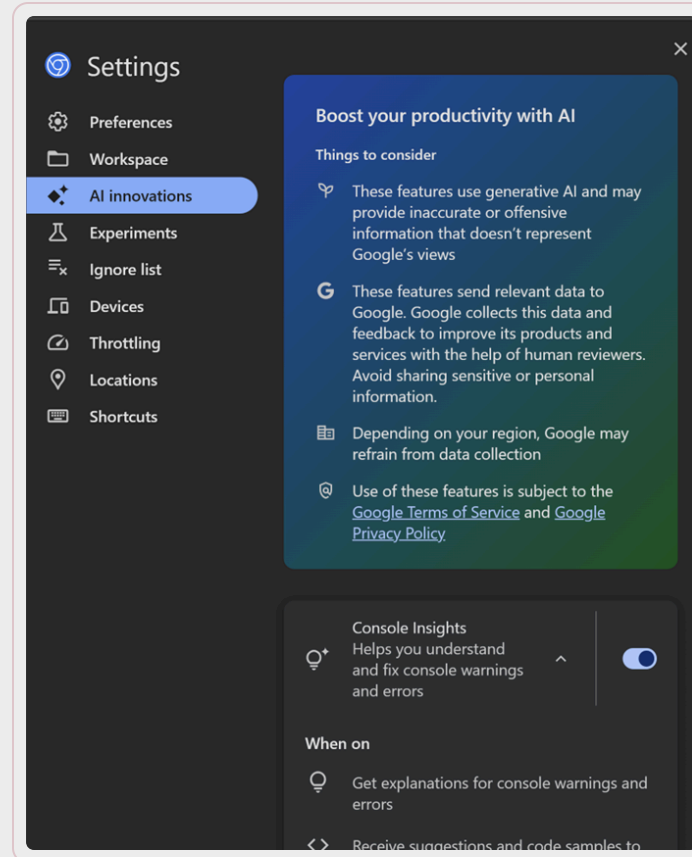
origin-when-cr

Analyzing network data

This request is for fetching the Google Tag Manager (GTM) JavaScript library, which is essential for implementing Google Analytics and other tracking functionalities on the web. The id=UA-165949399-1 parameter in the URL specifies the Google Analytics tracking ID.

Activación (Google Chrome)

Es necesario configurar el idioma de DevTools en inglés



HTTP: Request (header/body)

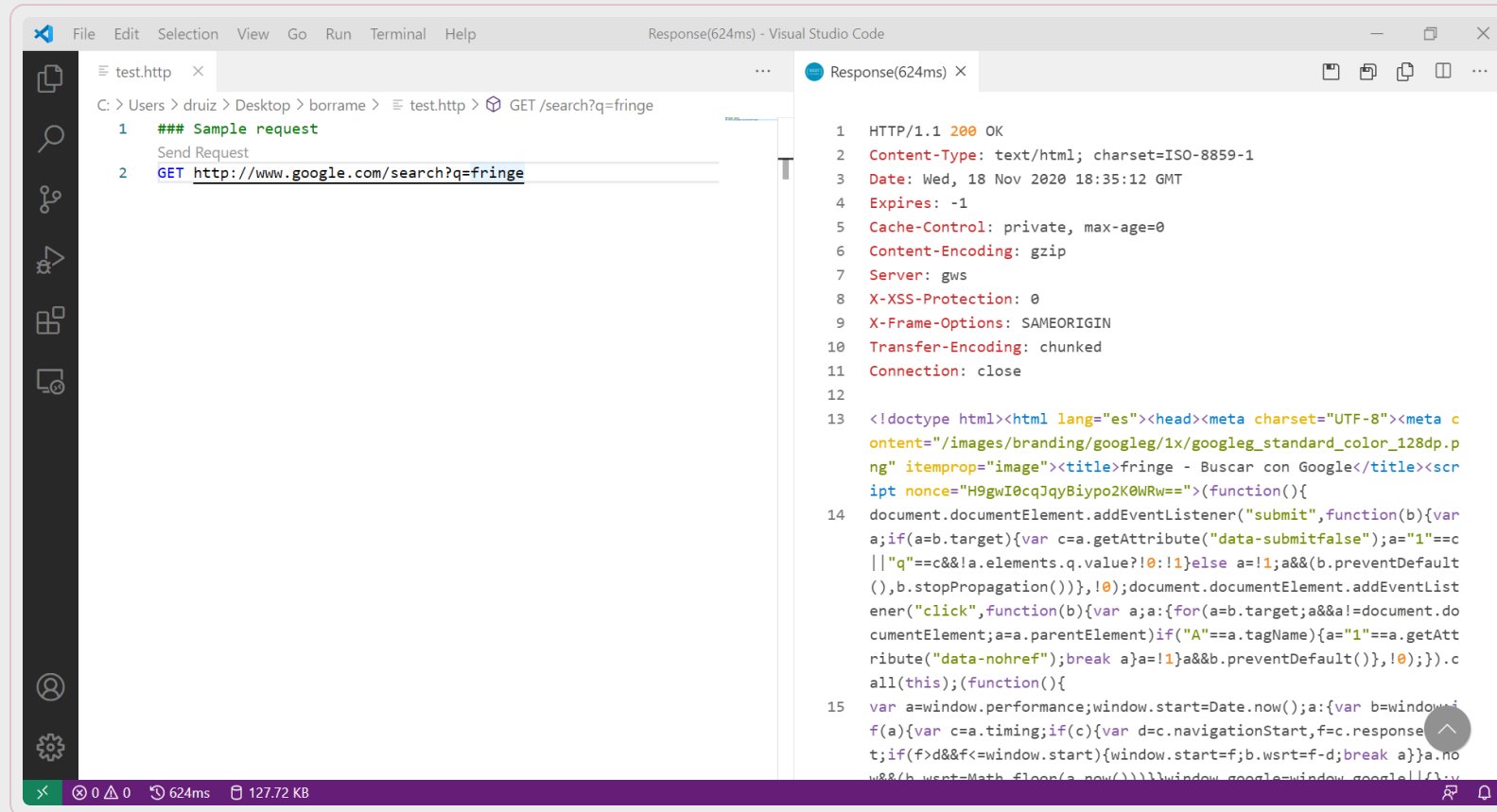
¿Qué formato tiene la petición (http request)?

```
GET HTTP/1.1
Host: www.google.com
User-Agent: PostmanRuntime/7.19.0
Accept: */*
Cache-Control: no-cache
Postman-Token: 02c2d340-3b50-4b3d-9f8e-df8dbd60f34b,d5a166b7-06c7-48ee-a9e1-f68fb7218742
Accept-Encoding: gzip, deflate
Cookie: 1P_JAR=2019-11-06-20; NID=190=Q00Cx_r_jNAtdYFzMS0v02mS5hAEGN47ddFMLJJT0vDo_D6EHdYKaKnK5evIwHQfGwi9cM9AjYIAJz-B3zUCw81RMPdQM_bZHZiU4U6rlljrbLZlp3bIsFtUx25-KqwFUSH1cJ-rm9g6TU_fnc0piiyxsT_UUdKvbz1E2azPdyI
Connection: keep-alive
cache-control: no-cache
```

```
GET HTTP/1.1
Host: www.us.es
User-Agent: PostmanRuntime/7.19.0
Accept: */*
Cache-Control: no-cache
Postman-Token: 76aa2e60-8fea-4edb-8e25-d4126943a1f8,903aa2ef-9c75-4033-9104-11bcd7265bb2
Accept-Encoding: gzip, deflate
Connection: keep-alive
cache-control: no-cache
```


HTTP: Response (header/body)

¿Qué formato tiene la respuesta (http response www.google.com)?



```
File Edit Selection View Go Run Terminal Help
Response(624ms) - Visual Studio Code

test.http x
C: > Users > drui > Desktop > borrame > test.http > GET /search?q=fringe
1 ### Sample request
   Send Request
2 GET http://www.google.com/search?q=fringe

Response(624ms) x
1 HTTP/1.1 200 OK
2 Content-Type: text/html; charset=ISO-8859-1
3 Date: Wed, 18 Nov 2020 18:35:12 GMT
4 Expires: -1
5 Cache-Control: private, max-age=0
6 Content-Encoding: gzip
7 Server: gws
8 X-XSS-Protection: 0
9 X-Frame-Options: SAMEORIGIN
10 Transfer-Encoding: chunked
11 Connection: close
12
13 <!doctype html><html lang="es"><head><meta charset="UTF-8"><meta c
   ontent="/images/branding/googleg/1x/googleg_standard_color_128dp.p
   ng" itemprop="image"><title>fringe - Buscar con Google</title><scr
   ipt nonce="H9gwI0cqJqyBiypo2K0WRw==">(function(){
14 document.documentElement.addEventListener("submit",function(b){var
   a;if(a=b.target){var c=a.getAttribute("data-submitfalse");a="1"==c
   ||"q"==c&&!a.elements.q.value?!0:!1}else a=!1;a&&(b.preventDefault
   (),b.stopPropagation()),!0);document.documentElement.addEventList
   ener("click",function(b){var a;a:{for(a=b.target;a&&a!=document.do
   cumentElement;a=a.parentElement)if("A"==a.tagName){a="1"==a.getAtt
   ribute("data-nohref");break a}a=!1;a&&b.preventDefault(),!0);}).c
   all(this);(function(){
15 var a=window.performance;window.start=Date.now();a:{var b=window.i
   f(a){var c=a.timing;if(c){var d=c.navigationStart,f=c.response
   t;if(f>d&&f<=window.start){window.start=f;b.wsr=f-d;break a}}a.no
   w&&(b.wsr=Math.floor(a.now()))}window.google=window.google||{};
```

HTTP: Response (header/body)

¿Qué formato tiene la respuesta (http response www.us.es)?

The screenshot displays the Visual Studio Code interface with a REST client extension. The left sidebar shows the Explorer view with a file named 'test.http'. The main editor area is split into two panes. The left pane shows the request details for a GET request to 'http://www.us.es'. The right pane shows the corresponding HTTP response.

Request Details:

- Method: GET
- URL: http://www.us.es

Response Details:

- Status: 200 OK
- Date: Wed, 18 Nov 2020 18:37:51 GMT
- X-Content-Type-Options: nosniff, nosniff
- Cache-Control: max-age=900, public
- X-Drupal-Dynamic-Cache: MISS
- Link: <http://www.us.es/>; rel="shortlink", <http://www.us.es/>; rel="canonical"
- X-UA-Compatible: IE=edge
- Content-language: es
- X-Frame-Options: SAMEORIGIN
- Expires: Sun, 19 Nov 1978 05:00:00 GMT
- Last-Modified: Wed, 18 Nov 2020 15:10:09 GMT
- Etag: "1605712209"
- Vary: Cookie, Accept-Encoding
- X-Drupal-Cache: HIT
- Content-Type: text/html; charset=UTF-8
- X-Varnish: 33044 98577
- Age: 25
- Connection: close
- Accept-Ranges: bytes
- Set-Cookie: BIGipServer~interna_1200~Pool_wm_us_es_https=rdio000000000000000000fffc0a81e44o80; path=/; Httponly; Secure
- Strict-Transport-Security: max-age=63072000; includeSubDomains
- Content-Encoding: gzip
- Transfer-Encoding: chunked

The response body contains HTML markup, including a DOCTYPE declaration, a head section with meta tags, and a script tag for Google Analytics.

HTTP: Response (header/body)

¿Qué formato tiene la respuesta (http response www.realbetisbalompie.es)?

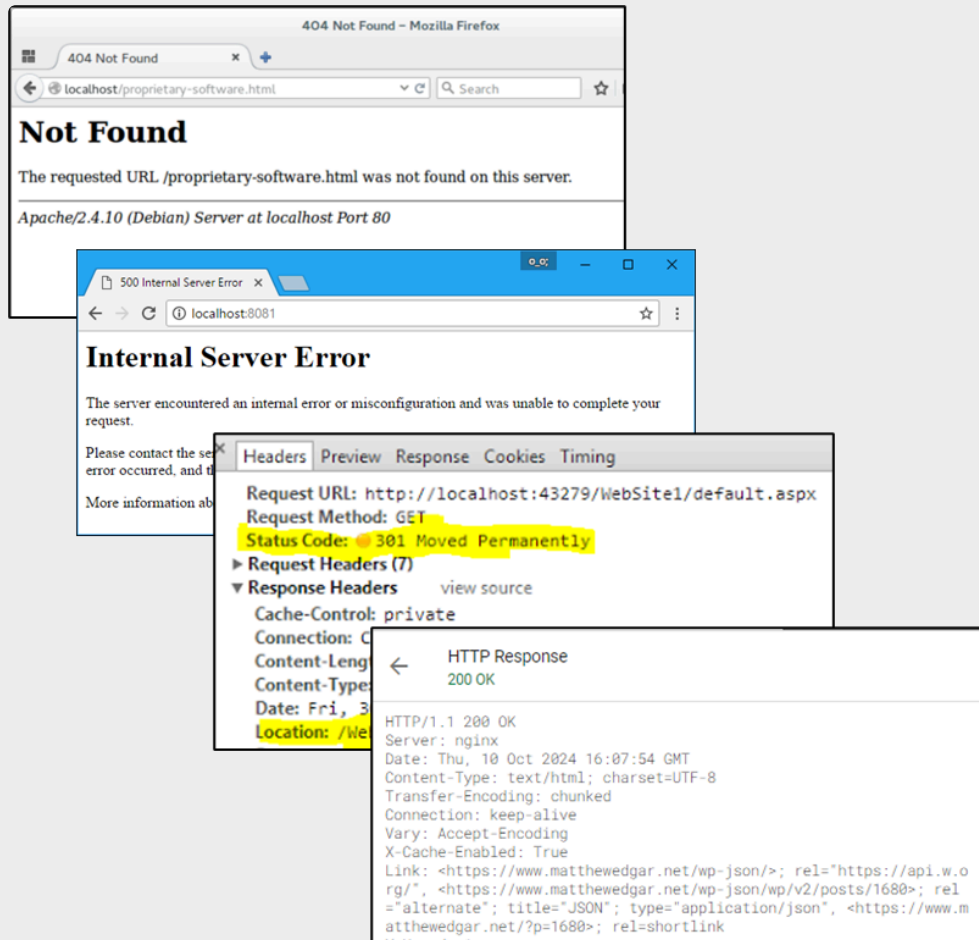
```
1  HTTP/1.1 200 OK
2  Server: nginx
3  Date: Mon, 26 Jan 2026 11:24:02 GMT
4  Content-Type: text/html; charset=UTF-8
5  Transfer-Encoding: chunked
6  Connection: close
7  Vary: Accept-Encoding
8  Strict-Transport-Security: max-age=31536000; includeSubDomains
9  X-Frame-Options: SAMEORIGIN
10 X-Content-Type-Options: nosniff
11 Permissions-Policy: camera=none, microphone=none, geolocation=none
12 Set-Cookie: PHPSESSID=jfk38ee1udv7vsc6gs8uselq11; path=/
13 Pragma: no-cache
14 Last-Modified: Mon, 26 Jan 2026 11:24:02 GMT
15 Expires: Mon, 26 Jan 2026 11:24:32 GMT
16 Cache-Control: max-age=30
17 Memcached: YES
18 Time: 2.4 ms
19 Memory: 741.88 kb
20 Content-Encoding: gzip
21
22 <meta name="csrf-token" content="596ef54c7c0322b8a81c543a7634200c5ad175304deba3ea1cabd5fba1904b4"><!
DOCTYPE html>
23 <html xmlns="http://www.w3.org/1999/xhtml" xmlns:fb="https://www.facebook.com/2008/fbml" xml:lang="es
-ES"
24     lang="es-ES">
```

HTTP: Códigos de respuesta

Códigos de respuesta HTTP

- 1XX: Informativa
- 2XX: Correcta
- 3XX: Redirección
- 4XX: Error del cliente
- 5XX: Error del servidor

https://es.wikipedia.org/wiki/Anexo:Códigos_de_estado_HTTP



HTTP: Códigos más comunes

https://en.wikipedia.org/wiki/List_of_HTTP_status_codes

Código	Nombre	Descripción
200	OK	Operación exitosa
201	Created	Recurso creado
204	No content	Operación exitosa pero no se devuelve nada
301	Moved permanently	El recurso se ha movido a otra URI permanentemente
307	Moved temporarily	El recurso se ha movido a otra URI temporalmente
400	Bad request	La petición no es válida
401	Unauthorized	El usuario no ha iniciado sesión
403	Forbidden	El usuario no tiene permisos
404	Not found	El recurso solicitado no existe
405	Method not allowed	El método HTTP no está permitido para ese recurso
500	Server error	El servidor ha tenido un error
503	Service unavailable	El servicio solicitado no se encuentra disponible

HTTP: Métodos

GET: Obtiene una representación del recurso especificado. (CRUD)

POST: Envía datos al servidor para su procesamiento. Puede resultar en la creación de nuevo recurso. (CRUD)

PUT: Actualiza totalmente el recurso especificado. (CRUD)

PATCH: Actualiza parcialmente el recurso especificado.(CRUD)

DELETE: Borra el recurso especificado. (CRUD)

HEAD: Pide una respuesta idéntica a la que correspondería a una petición GET, pero sin el cuerpo de la respuesta. Útil para comprobar si la caché sigue siendo válida mediante la cabecera Last-Modified.

OPTIONS: Devuelve los métodos HTTP que el servidor soporta para una URI.

https://es.wikipedia.org/wiki/Protocolo_de_transferencia_de_hipertexto#Métodos_de_petición

HTTP: Versiones

HTTP/0.9 (1991)

- Sólo implementa el comando GET

HTTP/1.0 (1996)

- GET, HEAD, POST

HTTP/1.1 (1999)

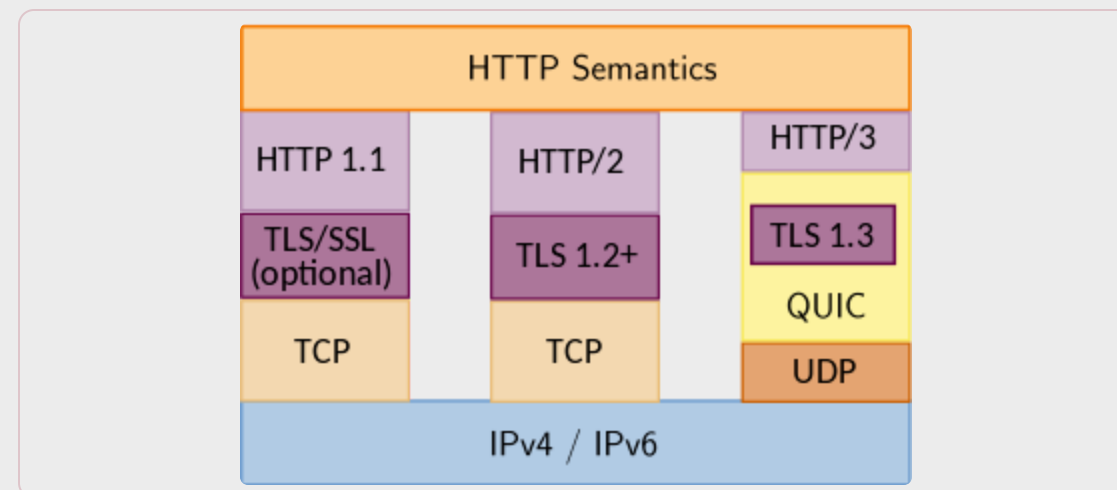
- GET, PUT, POST, DELETE
- Múltiples peticiones a la vez
- Sigue siendo el más usado hoy en día

HTTP/2.0 (2015)

- Mejora el empaquetado y transporte

HTTP/3.0 (2022 – Proposed standard)

- Basado en el protocolo QUIC propuesto por Google
- Usa UDP en lugar de TCP para aligerar las conexiones
- Aún no es estándar (implementado en el 95% de navegadores)



<https://en.wikipedia.org/wiki/HTTP/3>

Contenido

Introducción

URIs

Protocolo HTTP

Diseño de APIs REST

Consumir APIs REST

Diseño: Conceptos básicos

Los servicios web que implementan los principios de REST se denominan servicios RESTful. De igual forma, las APIs web implementadas a través de servicios RESTful reciben el nombre de APIs RESTful.

Un servicio RESTful permite interaccionar de forma programática con un recurso.

Los servicios RESTful implementan una o varias operaciones CRUD (Create, Retrieve, Update, Delete) sobre el recurso:

Operación	Método HTTP
Crear	POST
Consultar	GET
Actualizar	PUT / PATCH
Eliminar	DELETE

Diseño: Conceptos básicos

Departamentos (base http://ejemplo.es/api/v1)		
	URI	Descripción
GET	/departamentos	Obtiene todos los departamentos
GET	/departamentos/{id}	Obtiene el departamento con departmentold={id}
POST	/departamentos	Inserta un nuevo departamento con los valores indicados en el body de la petición
PUT	/departamentos/{id}	Actualiza el departamento con departmentold={id} con los valores indicados en el body de la petición
DELETE	/departamentos/{id}	Elimina el departamento con departmentold={id}

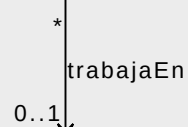
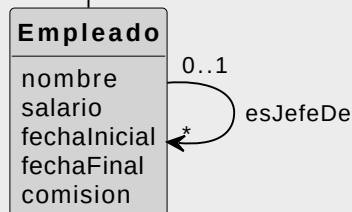
Diseño: Buenas prácticas

- Usar nombres en plural, no verbos
- Usar / para representar relaciones
- Usar versionado
- Usar diferentes métodos HTTP para diferentes acciones sobre el recurso
- Usar códigos de respuesta HTTP según el resultado de la operación
- Usar camelCase (https://en.wikipedia.org/wiki/Camel_case)
- Usar parámetros para filtrar, ordenar y paginar resultados



Diseño: Ejemplo. Trazabilidad

La fecha inicial no puede ser superior que la fecha final.
El nombre del empleado es único.
La comisión entre 0 y 1.



La combinación
(nombreDep, localidad)
no se puede repetir

Departamentos

PK(departamentoId)
AK(nombreDep, localidad)

departamentoId	nombreDep	localidad
----------------	-----------	-----------

Empleados

PK(empleadoId)
FK(departamentoId) / Departamentos
FK(jefeId) / Empleados

empleadoId	departamentoId	jefeId	nombre	salario	fechaInicial	fechaFinal	comision
------------	----------------	--------	--------	---------	--------------	------------	----------

Departamentos (base <http://ejemplo.es/api/v1>)

	URI	Descripción
GET	/departamentos	Obtiene todos los departamentos
GET	/departamentos/{id}	Obtiene el departamento con departamentoId={id}
POST	/departamentos	Inserta un nuevo departamento con los valores indicados en el body de la petición
PUT	/departamentos/{id}	Actualiza el departamento con departamentoId={id} con los valores indicados en el body de la petición
DELETE	/departamentos/{id}	Elimina el departamento con departamentoId={id}

Diseño: Ejemplo. Departamentos

Departamentos (base http://ejemplo.es/api/v1)		
	URI	Descripción
GET	/departamentos	Obtiene todos los departamentos
GET	/departamentos/{id}	Obtiene el departamento con departmentold={id}
POST	/departamentos	Inserta un nuevo departamento con los valores indicados en el body de la petición
PUT	/departamentos/{id}	Actualiza el departamento con departmentold={id} con los valores indicados en el body de la petición
DELETE	/departamentos/{id}	Elimina el departamento con departmentold={id}

Diseño: Ejemplo. Empleados

Empleados (base http://ejemplo.es/api/v1)		
Method	URI	Descripción
GET	/empleados	Obtiene todos los empleados
GET	/empleados?_sort={campo}	Obtiene los empleados ordenados por {campo}
GET	/empleados?_order={asc/desc}	Establece el orden como ascendente o descendente
GET	/empleados?_limit={n1}	Obtiene los n1 primeros empleados
GET	/empleados?_limit={n1}&_page={n2}	Obtiene los n1 empleados a partir de la página n2, con n1 empleados por página.
GET	/empleados/{id}	Obtiene el empleado con empleadold={id}
GET(*)	/departamentos/{id}/empleados	Obtiene los empleados del departamento con departamentold={id}
GET(*)	/empleados/{id}/jefe	Obtiene el jefe del empleado cuyo empleadold={id}
GET(*)	/departamentos/{id}/empleados?{atr1}={c1}&{atr2}={c2}...	Obtiene los empleados del departamento con departamentold={id} y cuyo valor para el atributo i-ésimo sea {ci}
POST	/empleados	Inserta un nuevo empleado con los valores indicados en el body de la petición
PUT	/empleados/id	Actualiza el empleado con empleadold={id} cambiando los valores indicados en el body de la petición
DELETE	/empleados/{id}	Elimina el empleado con empleadold={id}

Contenido

Introducción

URIs

Protocolo HTTP

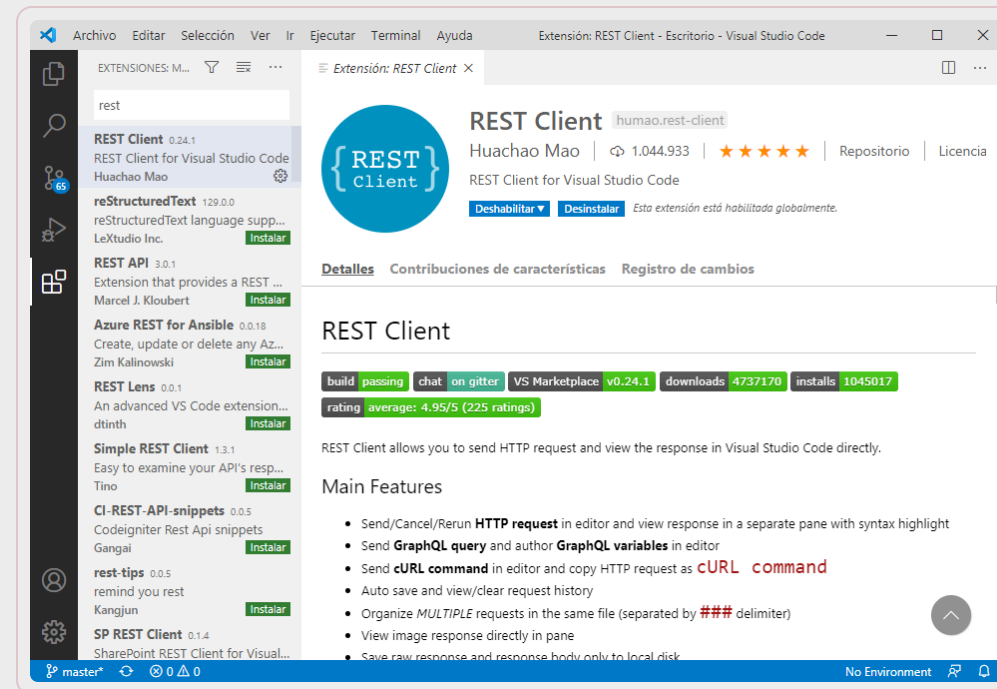
Diseño de APIs REST

Consumir APIs REST

Cientes REST: VS Code

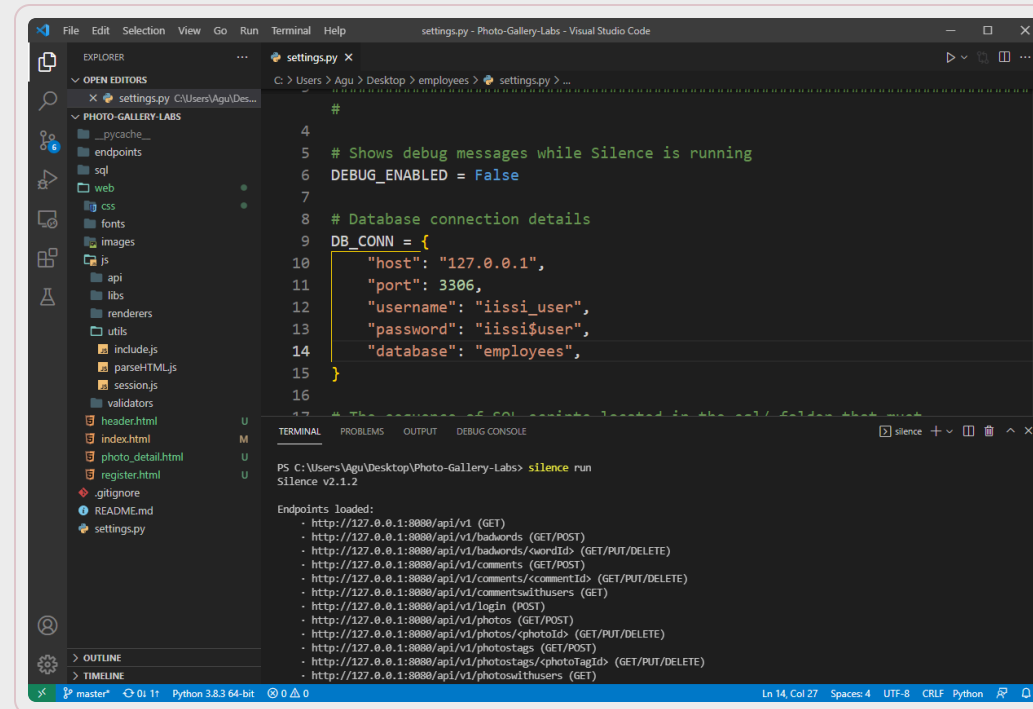
Extensión REST Client para VS Code

Permite hacer peticiones `HTTP` y ver sus respuestas desde el propio editor de código mediante archivos `.http`



Ejemplo de proyecto Silence

```
# Ejecuta estos comando en el terminal:  
> silence new employees --template Employees  
> silence createdb  
> silence createapi  
> silence run
```



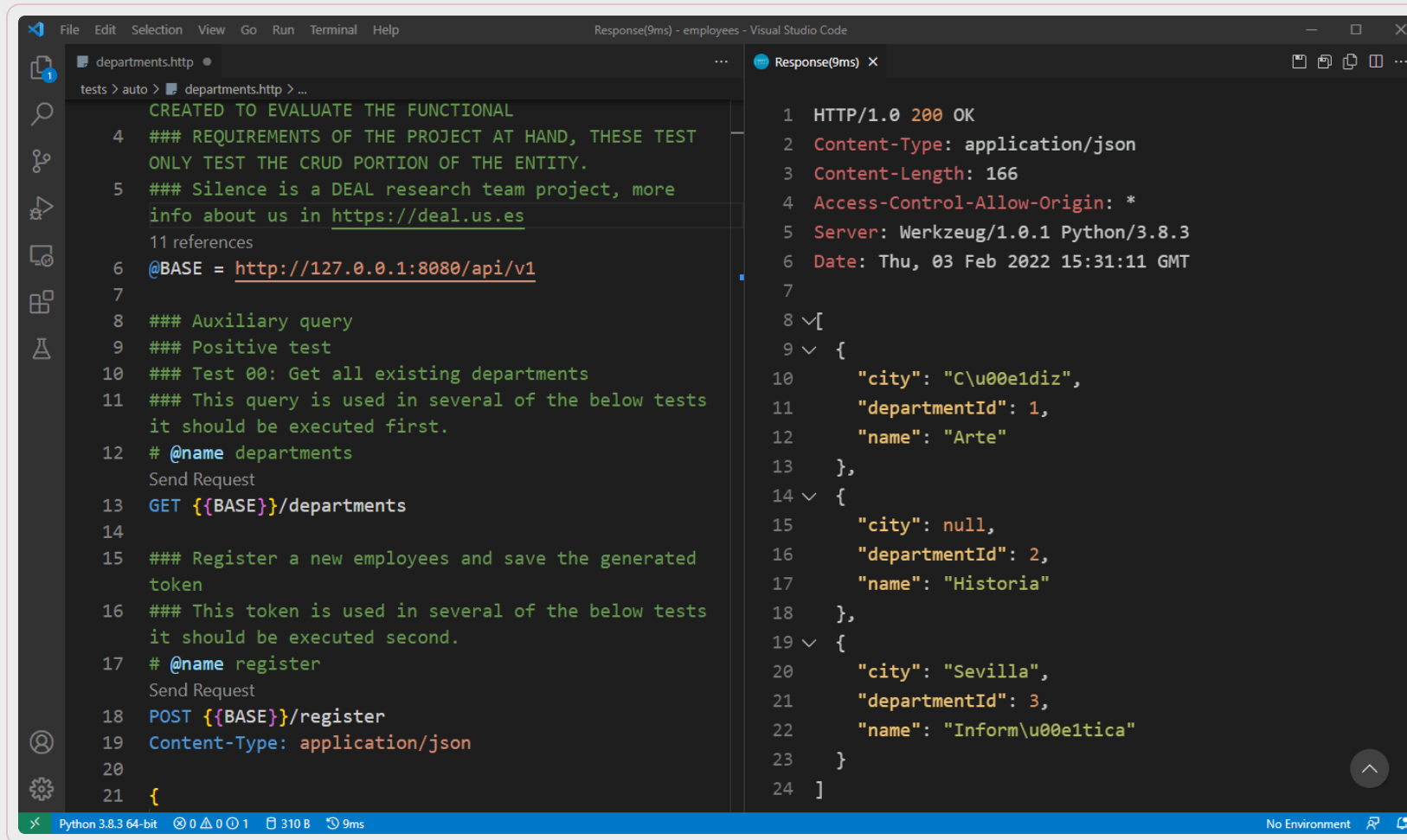
The screenshot shows the Visual Studio Code interface. On the left, the Explorer pane displays the project structure for 'PHOTO-GALLERY-LABS', including folders like 'api', 'lib', 'renderers', and 'utils', and files like 'header.html', 'index.html', 'photo_detail.html', and 'register.html'. The main editor area shows the 'settings.py' file with the following content:

```
#  
# Shows debug messages while Silence is running  
DEBUG_ENABLED = False  
  
# Database connection details  
DB_CONN = {  
    "host": "127.0.0.1",  
    "port": 3306,  
    "username": "iissi_user",  
    "password": "iissi$user",  
    "database": "employees",  
}  
  
# The names of SQL queries listed in the api folder that must
```

The bottom panel shows the Terminal with the command 'silence run' executed. The output indicates that the endpoints have been loaded successfully:

```
PS C:\Users\Agu\Desktop\Photo-Gallery-Labs> silence run  
Silence v2.1.2  
  
Endpoints loaded:  
- http://127.0.0.1:8080/api/v1 (GET)  
- http://127.0.0.1:8080/api/v1/badwords (GET/POST)  
- http://127.0.0.1:8080/api/v1/badwords/<wordId> (GET/PUT/DELETE)  
- http://127.0.0.1:8080/api/v1/comments (GET/POST)  
- http://127.0.0.1:8080/api/v1/comments/<commentId> (GET/PUT/DELETE)  
- http://127.0.0.1:8080/api/v1/commentswithusers (GET)  
- http://127.0.0.1:8080/api/v1/login (POST)  
- http://127.0.0.1:8080/api/v1/photos (GET/POST)  
- http://127.0.0.1:8080/api/v1/photos/<photoId> (GET/PUT/DELETE)  
- http://127.0.0.1:8080/api/v1/phototags (GET/POST)  
- http://127.0.0.1:8080/api/v1/phototags/<photoTagId> (GET/PUT/DELETE)  
- http://127.0.0.1:8080/api/v1/photoswithusers (GET)
```

Ejemplo: GET



The screenshot displays the Visual Studio Code interface with a REST client test file named `departments.http` and its corresponding response.

Left Panel (departments.http):

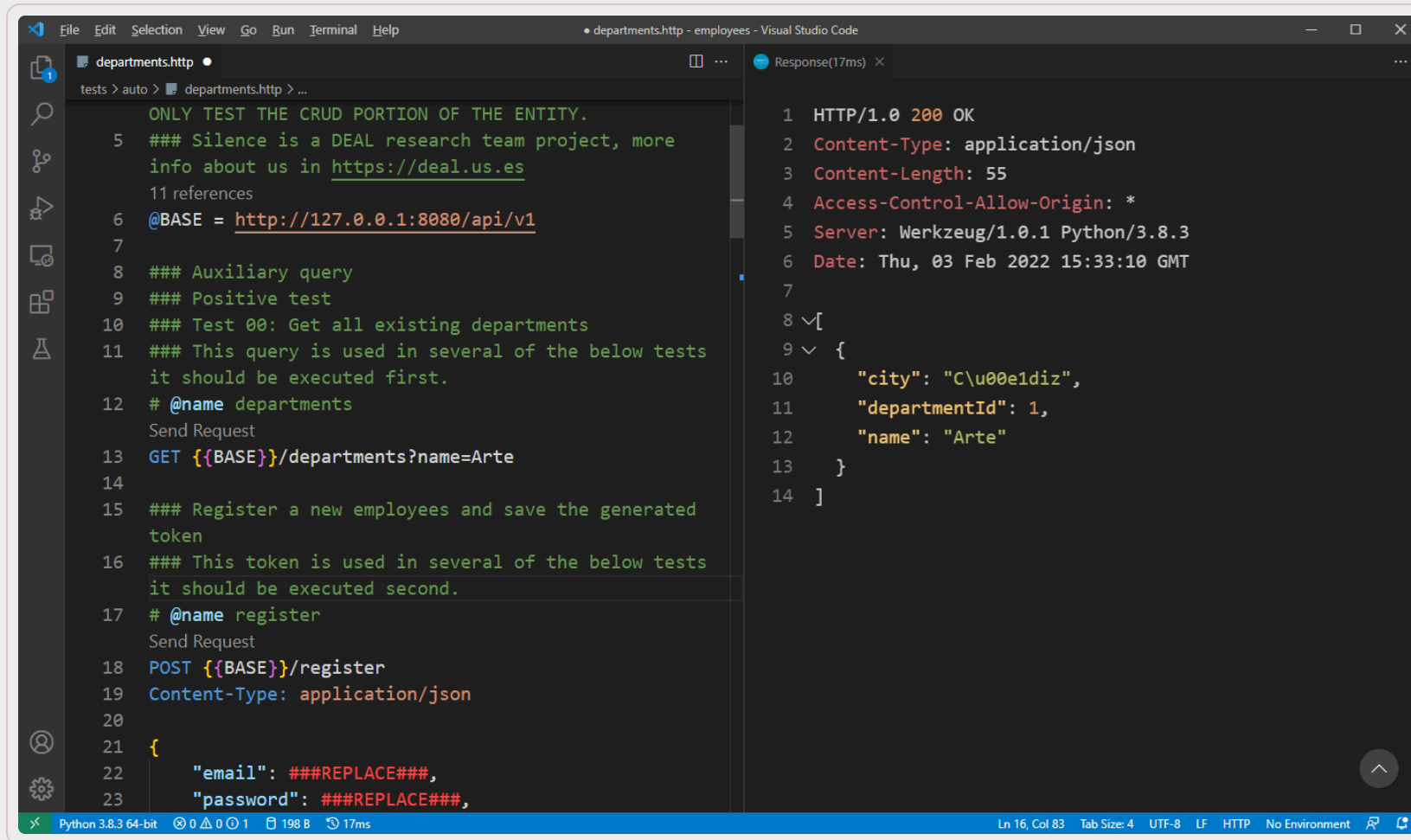
```
tests > auto > departments.http > ...
    CREATED TO EVALUATE THE FUNCTIONAL
4    ### REQUIREMENTS OF THE PROJECT AT HAND, THESE TEST
    ONLY TEST THE CRUD PORTION OF THE ENTITY.
5    ### Silence is a DEAL research team project, more
    info about us in https://deal.us.es
    11 references
6    @BASE = http://127.0.0.1:8080/api/v1
7
8    ### Auxiliary query
9    ### Positive test
10   ### Test 00: Get all existing departments
11   ### This query is used in several of the below tests
    it should be executed first.
12   # @name departments
    Send Request
13   GET {{BASE}}/departments
14
15   ### Register a new employees and save the generated
    token
16   ### This token is used in several of the below tests
    it should be executed second.
17   # @name register
    Send Request
18   POST {{BASE}}/register
19   Content-Type: application/json
20
21   {
```

Right Panel (Response(9ms)):

```
1  HTTP/1.0 200 OK
2  Content-Type: application/json
3  Content-Length: 166
4  Access-Control-Allow-Origin: *
5  Server: Werkzeug/1.0.1 Python/3.8.3
6  Date: Thu, 03 Feb 2022 15:31:11 GMT
7
8  [
9  {
10     "city": "C\u00e1diz",
11     "departmentId": 1,
12     "name": "Arte"
13  },
14  {
15     "city": null,
16     "departmentId": 2,
17     "name": "Historia"
18  },
19  {
20     "city": "Sevilla",
21     "departmentId": 3,
22     "name": "Inform\u00e1tica"
23  }
24  ]
```

The status bar at the bottom indicates the environment is `No Environment` and the response time is `9ms`.

Ejemplo: GET con variable



The screenshot displays the Visual Studio Code interface with a file named `departments.http` open. The editor shows a series of comments and a REST client request. The request is a GET call to `http://127.0.0.1:8080/api/v1/departments?name=Arte`. To the right, the 'Response' pane shows the server's reply, which is a 200 OK status with a JSON body containing department information.

```
File Edit Selection View Go Run Terminal Help
• departments.http - employees - Visual Studio Code

tests > auto > departments.http > ...

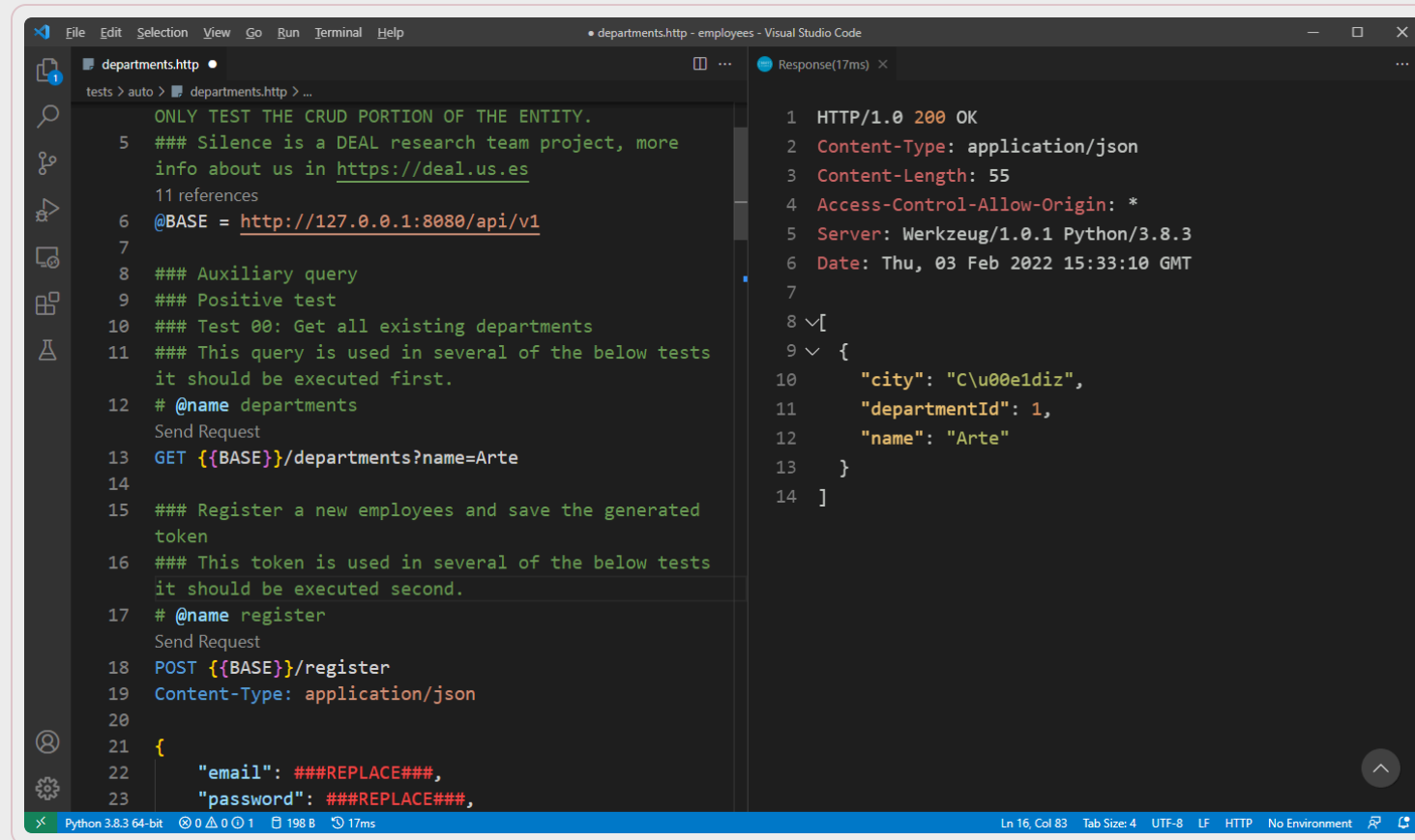
ONLY TEST THE CRUD PORTION OF THE ENTITY.
5  ### Silence is a DEAL research team project, more
   info about us in https://deal.us.es
   11 references
6  @BASE = http://127.0.0.1:8080/api/v1
7
8  ### Auxiliary query
9  ### Positive test
10 ### Test 00: Get all existing departments
11 ### This query is used in several of the below tests
   it should be executed first.
12 # @name departments
   Send Request
13 GET {{BASE}}/departments?name=Arte
14
15 ### Register a new employees and save the generated
   token
16 ### This token is used in several of the below tests
   it should be executed second.
17 # @name register
   Send Request
18 POST {{BASE}}/register
19 Content-Type: application/json
20
21 {
22   "email": ###REPLACE###,
23   "password": ###REPLACE###,
```

```
1 HTTP/1.0 200 OK
2 Content-Type: application/json
3 Content-Length: 55
4 Access-Control-Allow-Origin: *
5 Server: Werkzeug/1.0.1 Python/3.8.3
6 Date: Thu, 03 Feb 2022 15:33:10 GMT
7
8 [
9   {
10     "city": "C\u00e1diz",
11     "departmentId": 1,
12     "name": "Arte"
13   }
14 ]
```

Python 3.8.3 64-bit 0 0 1 198 B 17ms Ln 16, Col 83 Tab Size: 4 UTF-8 LF HTTP No Environment

Ejemplo: GET con parámetros

Los parámetros van en la query string



The screenshot shows the Visual Studio Code interface with an HTTP client extension. The left pane displays a test script for a REST API, and the right pane shows the response to a GET request.

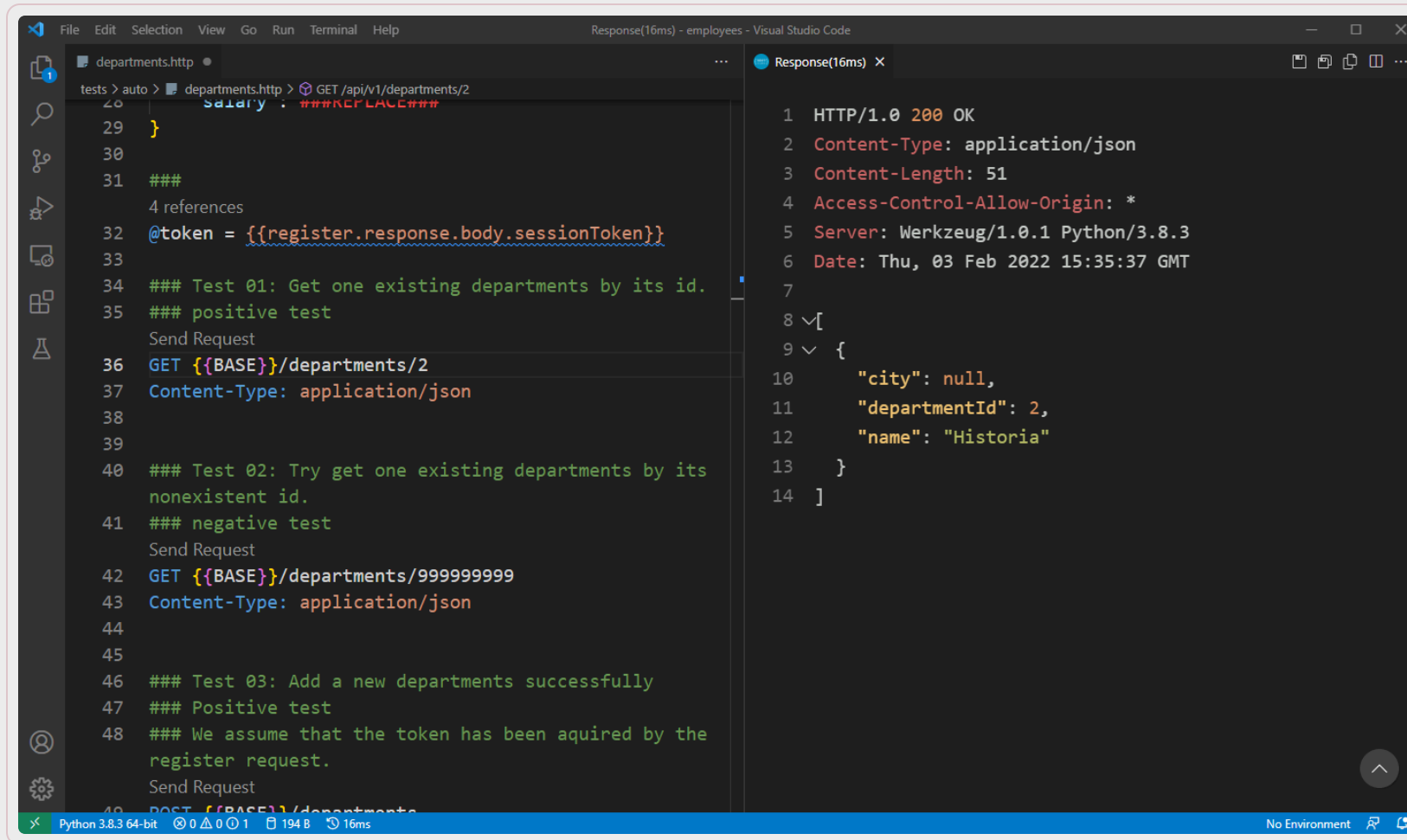
Test Script (departments.http):

```
ONLY TEST THE CRUD PORTION OF THE ENTITY.
5  ### Silence is a DEAL research team project, more
   info about us in https://deal.us.es
   11 references
6  @BASE = http://127.0.0.1:8080/api/v1
7
8  ### Auxiliary query
9  ### Positive test
10 ### Test 00: Get all existing departments
11 ### This query is used in several of the below tests
   it should be executed first.
12 # @name departments
   Send Request
13 GET {{BASE}}/departments?name=Arte
14
15 ### Register a new employees and save the generated
   token
16 ### This token is used in several of the below tests
   it should be executed second.
17 # @name register
   Send Request
18 POST {{BASE}}/register
19 Content-Type: application/json
20
21 {
22   "email": ###REPLACE###,
23   "password": ###REPLACE###,
```

Response (17ms):

```
1 HTTP/1.0 200 OK
2 Content-Type: application/json
3 Content-Length: 55
4 Access-Control-Allow-Origin: *
5 Server: Werkzeug/1.0.1 Python/3.8.3
6 Date: Thu, 03 Feb 2022 15:33:10 GMT
7
8 [
9   {
10     "city": "C\u00e1diz",
11     "departmentId": 1,
12     "name": "Arte"
13   }
14 ]
```

Ejemplo: GET por ID



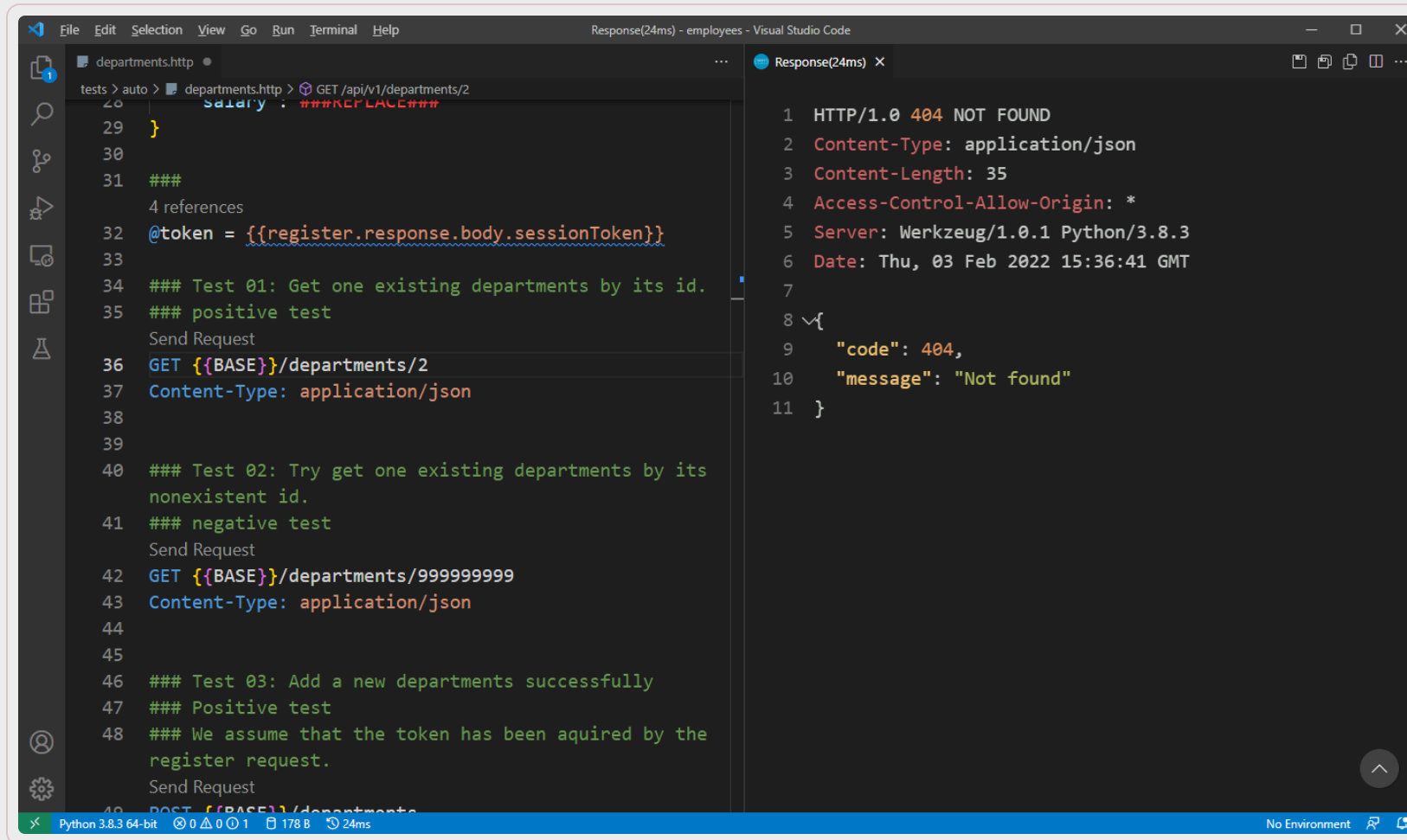
```
File Edit Selection View Go Run Terminal Help
Response(16ms) - employees - Visual Studio Code

departments.http
tests > auto > departments.http > GET /api/v1/departments/2
28 salary: ###REPLACE###
29 }
30
31 ###
32 4 references
33 @token = {{register.response.body.sessionToken}}
34
35 ### Test 01: Get one existing departments by its id.
36 ### positive test
37 Send Request
38 GET {{BASE}}/departments/2
39 Content-Type: application/json
40
41 ### Test 02: Try get one existing departments by its
42 nonexistent id.
43 ### negative test
44 Send Request
45 GET {{BASE}}/departments/999999999
46 Content-Type: application/json
47
48 ### Test 03: Add a new departments successfully
49 ### Positive test
50 ### We assume that the token has been acquired by the
51 register request.
52 Send Request
53 POST {{BASE}}/departments/

Response(16ms) X
1 HTTP/1.0 200 OK
2 Content-Type: application/json
3 Content-Length: 51
4 Access-Control-Allow-Origin: *
5 Server: Werkzeug/1.0.1 Python/3.8.3
6 Date: Thu, 03 Feb 2022 15:35:37 GMT
7
8 [
9   {
10     "city": null,
11     "departmentId": 2,
12     "name": "Historia"
13   }
14 ]

Python 3.8.3 64-bit 0 0 0 1 194 B 16ms No Environment
```

Ejemplo: GET por ID



The screenshot shows the Visual Studio Code interface with a REST client file named `departments.http` and its corresponding response.

Left Panel (departments.http):

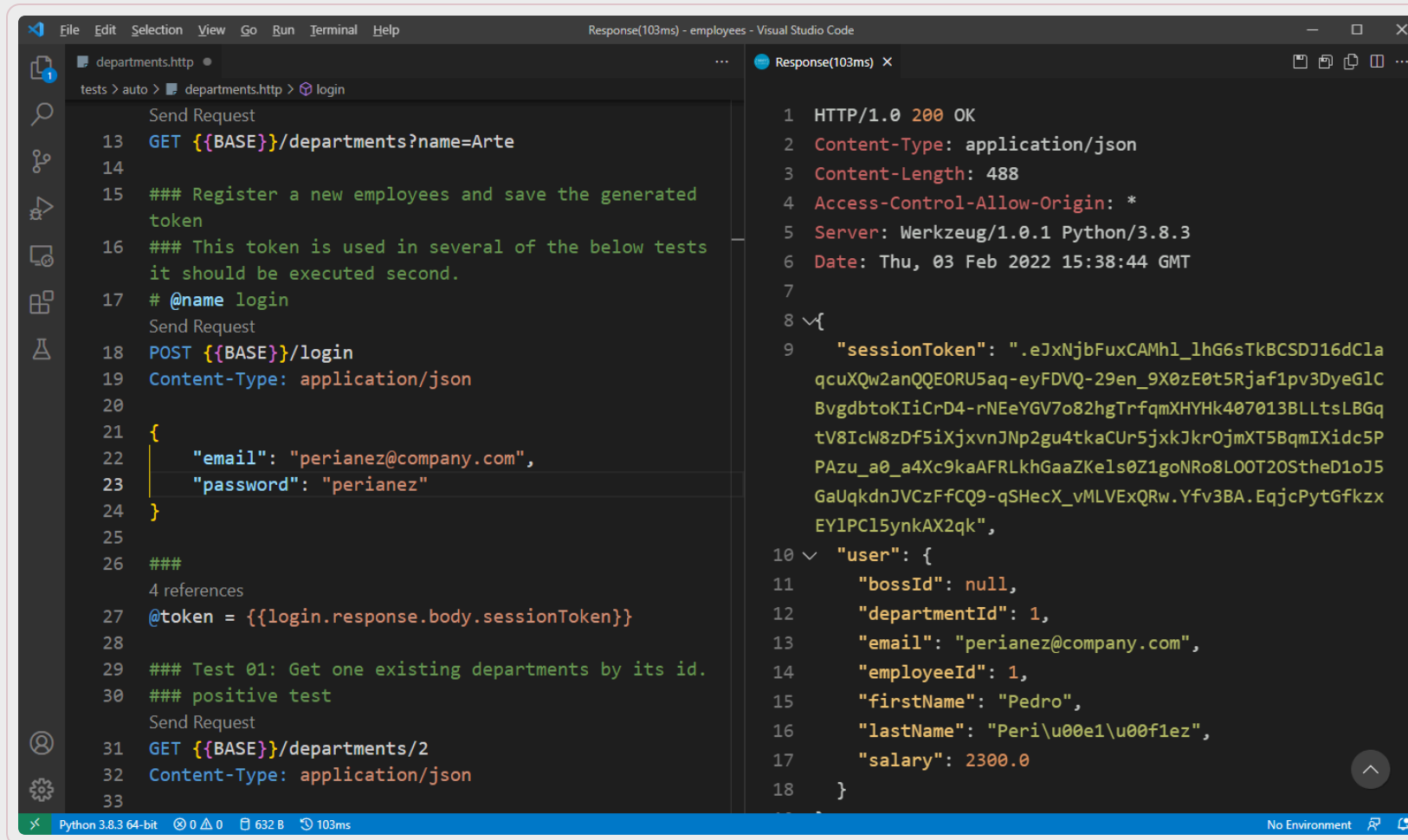
```
tests > auto > departments.http > GET /api/v1/departments/2
28 salary: ###REPLACE###
29 }
30
31 ###
32 4 references
33 @token = {{register.response.body.sessionToken}}
34
35 ### Test 01: Get one existing departments by its id.
36 ### positive test
37 Send Request
38 GET {{BASE}}/departments/2
39 Content-Type: application/json
40
41 ### Test 02: Try get one existing departments by its
42 nonexistent id.
43 ### negative test
44 Send Request
45 GET {{BASE}}/departments/999999999
46 Content-Type: application/json
47
48 ### Test 03: Add a new departments successfully
49 ### Positive test
50 ### We assume that the token has been acquired by the
51 register request.
52 Send Request
53 POST {{BASE}}/departments
```

Right Panel (Response(24ms)):

```
1 HTTP/1.0 404 NOT FOUND
2 Content-Type: application/json
3 Content-Length: 35
4 Access-Control-Allow-Origin: *
5 Server: Werkzeug/1.0.1 Python/3.8.3
6 Date: Thu, 03 Feb 2022 15:36:41 GMT
7
8 {
9   "code": 404,
10  "message": "Not found"
11 }
```

Bottom Status Bar: Python 3.8.3 64-bit, 0 0 1, 178 B, 24ms, No Environment

Ejemplo: POST – Solicitar token



```
File Edit Selection View Go Run Terminal Help
Response(103ms) - employees - Visual Studio Code

departments.http
tests > auto > departments.http > login

Send Request
13 GET {{BASE}}/departments?name=Arte
14
15 ### Register a new employees and save the generated token
16 ### This token is used in several of the below tests it should be executed second.
17 # @name login
Send Request
18 POST {{BASE}}/login
19 Content-Type: application/json
20
21 {
22   "email": "perianez@company.com",
23   "password": "perianez"
24 }
25
26 ###
27 4 references
28 @token = {{login.response.body.sessionToken}}
29 ### Test 01: Get one existing departments by its id.
30 ### positive test
Send Request
31 GET {{BASE}}/departments/2
32 Content-Type: application/json
33

Response(103ms) X
1 HTTP/1.0 200 OK
2 Content-Type: application/json
3 Content-Length: 488
4 Access-Control-Allow-Origin: *
5 Server: Werkzeug/1.0.1 Python/3.8.3
6 Date: Thu, 03 Feb 2022 15:38:44 GMT
7
8 {
9   "sessionToken": ".eJxNjbFuxCAMh1_lhG6sTkBCSDJ16dC1a
qcuXQw2anQQEORU5aq-eyFDVQ-29en_9X0zE0t5Rjaf1pv3DyeG1C
BvgdbtoKIiCrD4-rNEeYGV7o82hgTrfqmXHYHk407013BLltsLBGq
tV8IcW8zDf5iXjxvnJNp2gu4tkaCUr5jxkJkrOjmXT5BqmIXidc5P
PAzu_a0_a4Xc9kaAFRLkhGaaZKels0Z1goNRo8LOOT20StheD1oJ5
GaUqkdnJVCzFfCQ9-qSHecX_vMLVExQRw.Yfv3BA.EqjcPytGfkzx
EY1PC15ynkAX2qk",
10  "user": {
11    "bossId": null,
12    "departmentId": 1,
13    "email": "perianez@company.com",
14    "employeeId": 1,
15    "firstName": "Pedro",
16    "lastName": "Peri\u00e1\u00f1ez",
17    "salary": 2300.0
18  }
```

Python 3.8.3 64-bit 0 0 632 B 103ms No Environment

Ejemplo: POST - Insertar

The screenshot shows the Visual Studio Code interface with an HTTP client extension. The left pane displays a test script for a POST request to insert a new department. The right pane shows the response of the request.

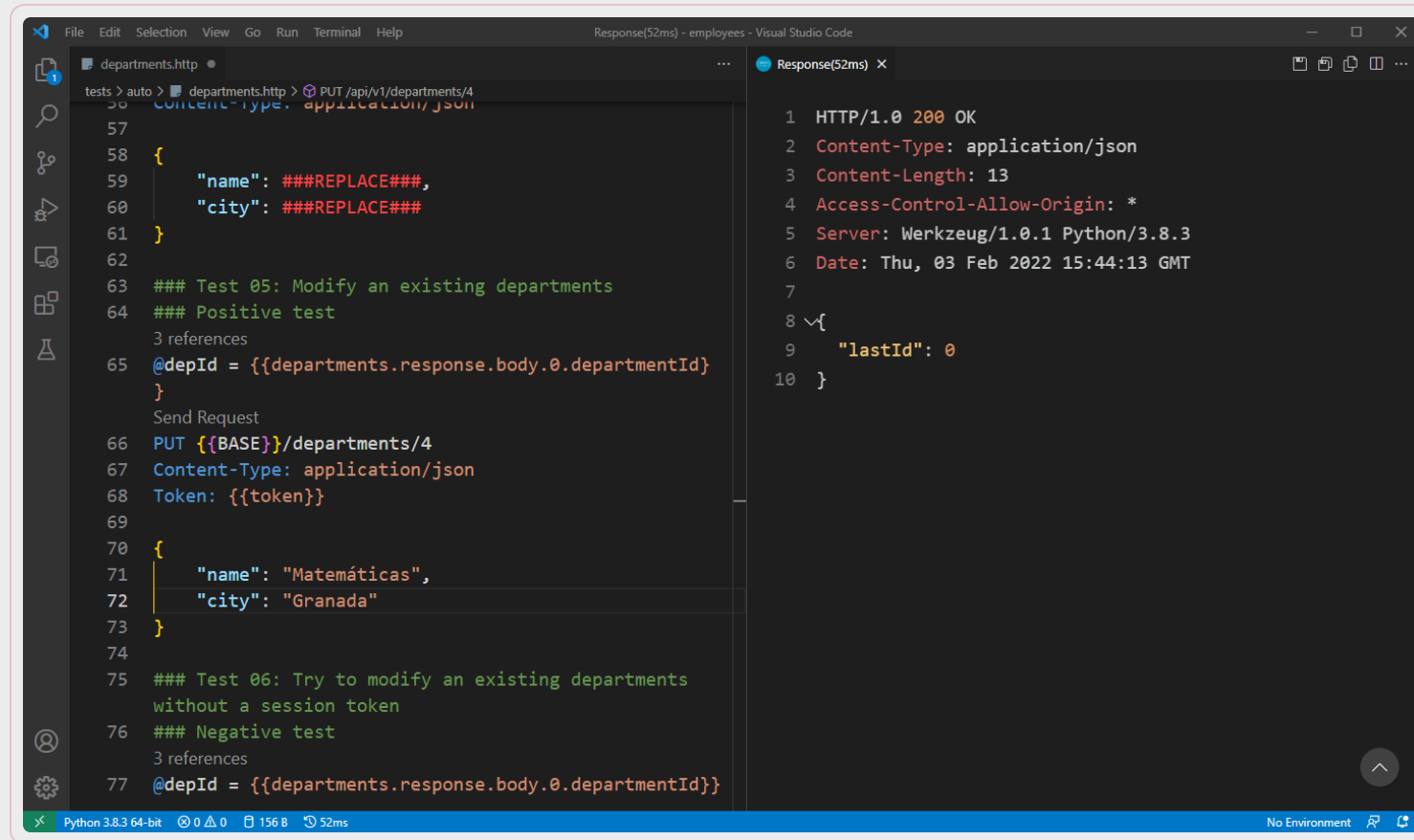
```
35  ### Test 02: Try get one existing departments by its
36  nonexistent id.
37  ### negative test
38  Send Request
39  GET {{BASE}}/departments/999999999
40  Content-Type: application/json
41  ### Test 03: Add a new departments successfully
42  ### Positive test
43  ### We assume that the token has been aquired by the
44  register request.
45  Send Request
46  POST {{BASE}}/departments
47  Content-Type: application/json
48  Token: {{token}}
49  {
50    "name": "Matemáticas",
51    "city": "Málaga"
52  }
53  ### Test 04: Add a new departments without a session
54  token
55  ### Negative test
56  Send Request
57  POST {{BASE}}/departments
```

The response pane shows the following details:

```
1  HTTP/1.0 200 OK
2  Content-Type: application/json
3  Content-Length: 13
4  Access-Control-Allow-Origin: *
5  Server: Werkzeug/1.0.1 Python/3.8.3
6  Date: Thu, 03 Feb 2022 15:43:13 GMT
7
8  {
9    "lastId": 4
10 }
```

The status bar at the bottom indicates: Python 3.8.3 64-bit, 0 errors, 0 warnings, 156 B, 93ms. The bottom right corner shows: Ln 53, Col 59, Tab Size: 4, UTF-8, LF, HTTP, No Environment.

Ejemplo: PUT - Actualizar



The screenshot shows the Visual Studio Code interface with a REST client test in a file named `departments.http`. The test is for a PUT request to `PUT /api/v1/departments/4` with a JSON body. The response is shown in the `Response(52ms)` panel.

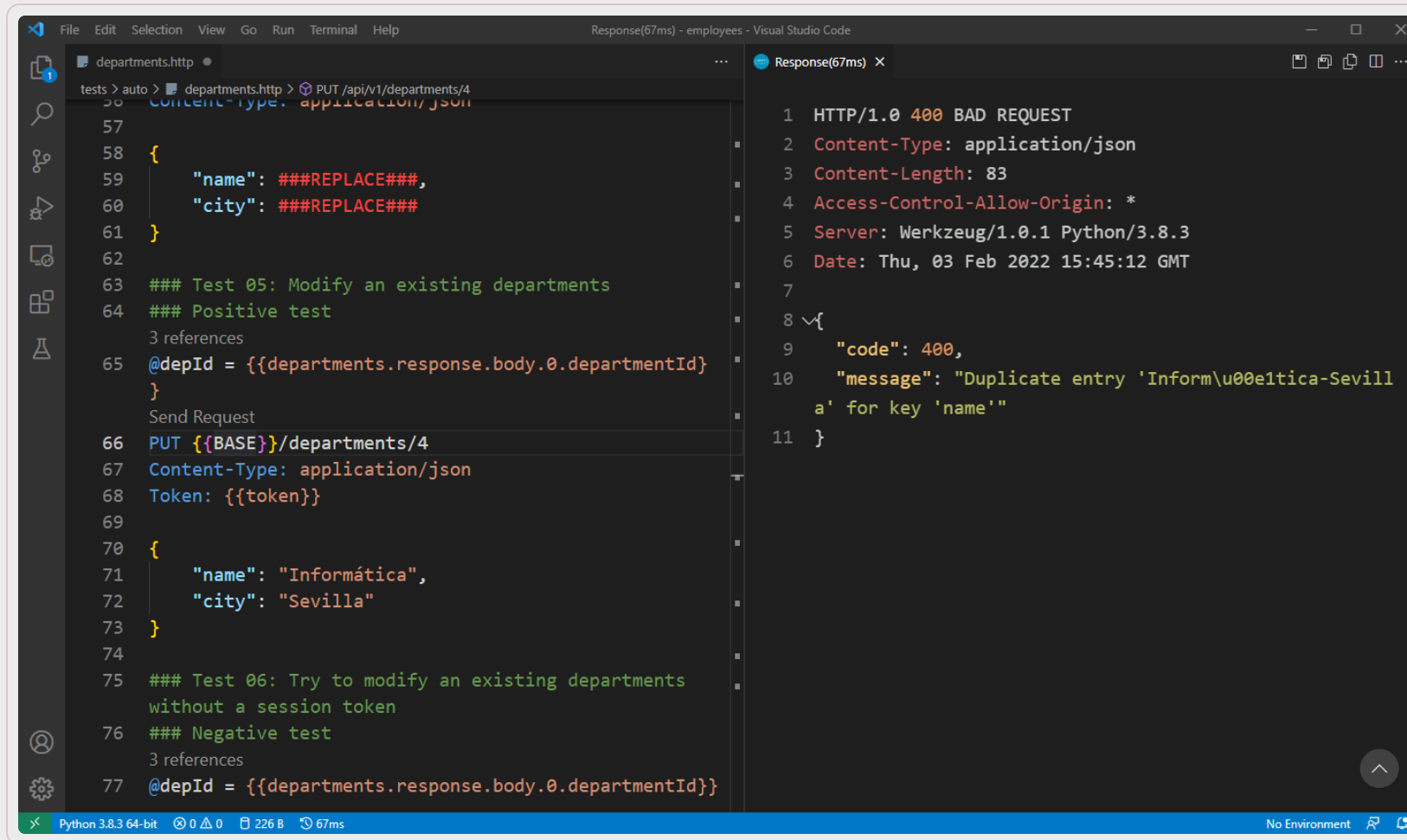
```
56 tests > auto > departments.http > PUT /api/v1/departments/4
57 Content-Type: application/json
58 {
59   "name": ###REPLACE###,
60   "city": ###REPLACE###
61 }
62
63 ### Test 05: Modify an existing departments
64 ### Positive test
65 3 references
66 @depId = {{departments.response.body.0.departmentId}}
67 Send Request
68 PUT {{BASE}}/departments/4
69 Content-Type: application/json
70 Token: {{token}}
71
72 {
73   "name": "Matemáticas",
74   "city": "Granada"
75 }
76
77 ### Test 06: Try to modify an existing departments
78 without a session token
79 ### Negative test
80 3 references
81 @depId = {{departments.response.body.0.departmentId}}
```

The response in the `Response(52ms)` panel is:

```
1 HTTP/1.0 200 OK
2 Content-Type: application/json
3 Content-Length: 13
4 Access-Control-Allow-Origin: *
5 Server: Werkzeug/1.0.1 Python/3.8.3
6 Date: Thu, 03 Feb 2022 15:44:13 GMT
7
8 {
9   "lastId": 0
10 }
```

Limitación de MySQL/MariaDB, devuelve lastId=0 cuando se actualiza

Ejemplo: PUT - Actualizar



The screenshot shows a Visual Studio Code window with a REST client test in the left pane and its response in the right pane.

Left Pane (departments.http):

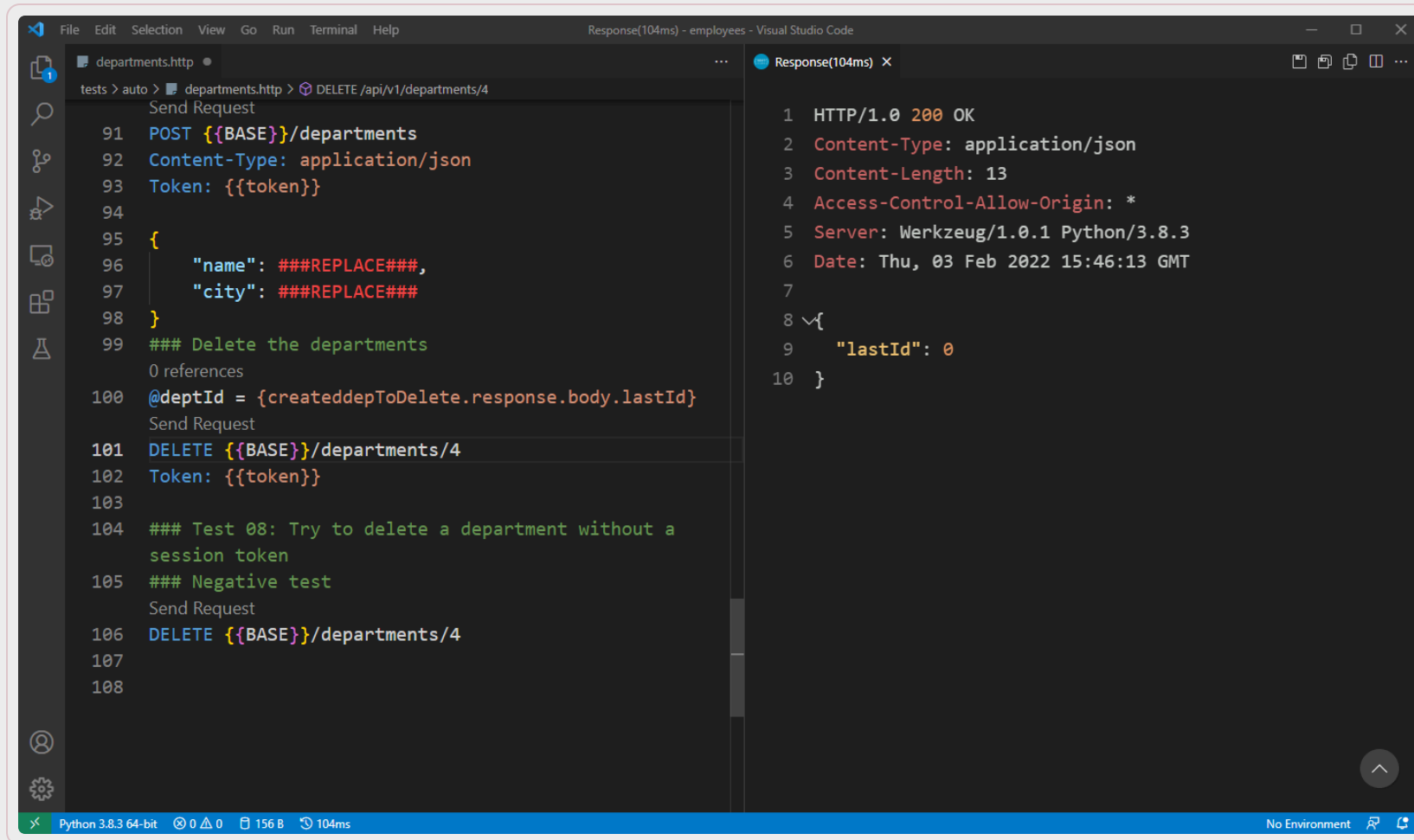
```
tests > auto > departments.http > PUT /api/v1/departments/4
56 Content-Type: application/json
57
58 {
59     "name": ###REPLACE###,
60     "city": ###REPLACE###
61 }
62
63 ### Test 05: Modify an existing departments
64 ### Positive test
65 @depId = {{departments.response.body.0.departmentId}}
66 PUT {{BASE}}/departments/4
67 Content-Type: application/json
68 Token: {{token}}
69
70 {
71     "name": "Informática",
72     "city": "Sevilla"
73 }
74
75 ### Test 06: Try to modify an existing departments
76 ### Negative test
77 @depId = {{departments.response.body.0.departmentId}}
```

Right Pane (Response(67ms) X):

```
1 HTTP/1.0 400 BAD REQUEST
2 Content-Type: application/json
3 Content-Length: 83
4 Access-Control-Allow-Origin: *
5 Server: Werkzeug/1.0.1 Python/3.8.3
6 Date: Thu, 03 Feb 2022 15:45:12 GMT
7
8 {
9     "code": 400,
10    "message": "Duplicate entry 'Inform\u00e1tica-Sevilla' for key 'name'"
11 }
```

The status bar at the bottom indicates: Python 3.8.3 64-bit, 0 errors, 226 B, 67ms, No Environment.

Ejemplo: DELETE



```
File Edit Selection View Go Run Terminal Help
Response(104ms) - employees - Visual Studio Code

departments.http
tests > auto > departments.http > DELETE /api/v1/departments/4
Send Request
91 POST {{BASE}}/departments
92 Content-Type: application/json
93 Token: {{token}}
94
95 {
96   "name": ###REPLACE###,
97   "city": ###REPLACE###
98 }
99 ### Delete the departments
0 references
100 @deptId = {createddepToDelete.response.body.lastId}
Send Request
101 DELETE {{BASE}}/departments/4
102 Token: {{token}}
103
104 ### Test 08: Try to delete a department without a
    session token
105 ### Negative test
Send Request
106 DELETE {{BASE}}/departments/4
107
108

Response(104ms) X
1 HTTP/1.0 200 OK
2 Content-Type: application/json
3 Content-Length: 13
4 Access-Control-Allow-Origin: *
5 Server: Werkzeug/1.0.1 Python/3.8.3
6 Date: Thu, 03 Feb 2022 15:46:13 GMT
7
8 {
9   "lastId": 0
10 }
```

Python 3.8.3 64-bit 0 0 156 B 104ms No Environment



Gracias

Universidad de Sevilla

**Departamento de Lenguajes y Sistemas
Informáticos**

UNIVERSIDAD DE SEVILLA